Compiler Design

Unit 1: Overview of Compilation

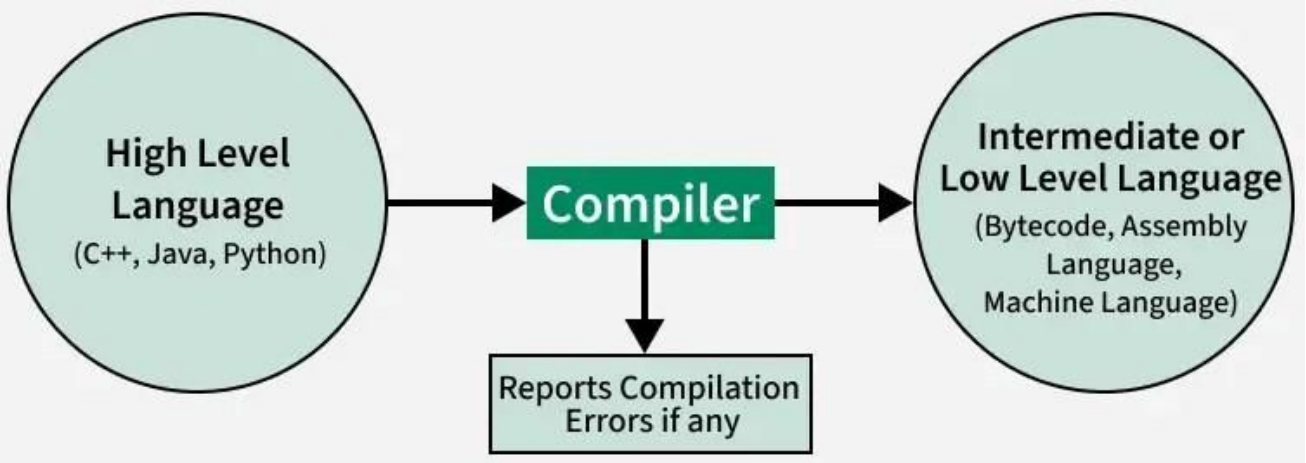
4 Hrs

1.1 Compiler and Interpreter
1.2 Compiler Structure
1.3 Overview of Translation
    1.3.1 The Front End
    1.3.2 The Optimizer
    1.3.3 The Backend

Introduction

What is a Compiler?

A compiler is a software. This is basically a special program that takes the code as input. The code is generally written in a high-level language and turns it into machine language that a computer can run. It acts as a translator, reading the source code, and producing optimized machine code.



Why can’t computer understand high level language?

High-level languages use English-like syntax and concepts (loops, functions, objects). The computer’s CPU doesn’t understand words like if, while, or print.

Computers use binary (0 and 1) not because it’s the only possible system, but because it’s the most reliable and practical for electronic hardware.

1. Built from electronic switches
 At the lowest level, a computer is made of tiny switches called transistors (used in chips like those designed by Intel).
 Each transistor has two stable states:

- ON → 1
- OFF → 0

It’s very natural to represent data using just these two states.

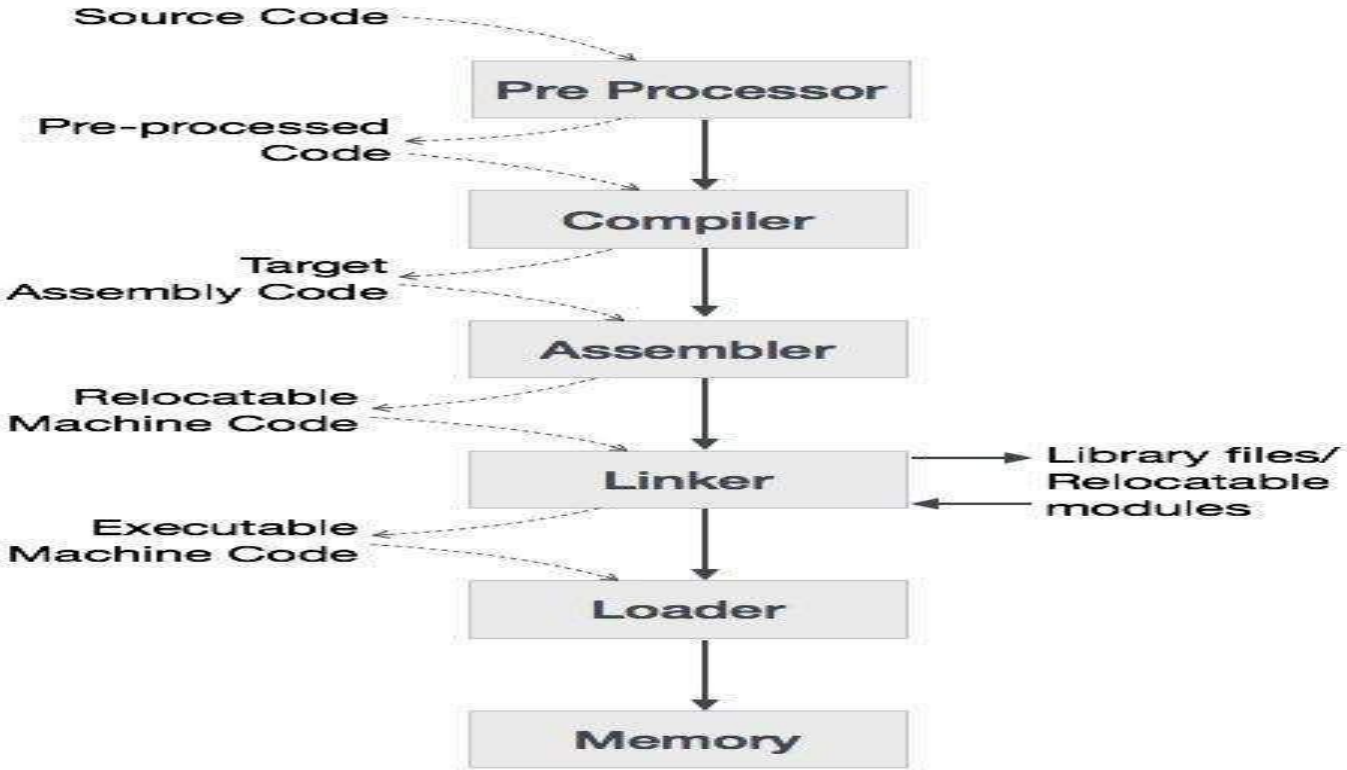
2. Reliability (very important)
Electronic signals aren’t perfectly clean—there’s always noise.
With only two states:

- Easy to distinguish (high voltage vs low voltage)
- Less chance of error

If we used more levels (like 0–9), small voltage changes could easily cause mistakes.

3. Simpler circuit design
Binary makes logic circuits (AND, OR, NOT gates) simple and efficient.
These circuits are based on a concept from Boolean Algebra, which works perfectly with true/false (1/0).

It goes through several stages, like **scanning, parsing, and checking** meanings.This is for checking whether everything is correct or not. And, it makes sure that the original logic is preserved. It is a fundamental tool for software development, and it makes computers to understand and execute our instructions written in higher level language.

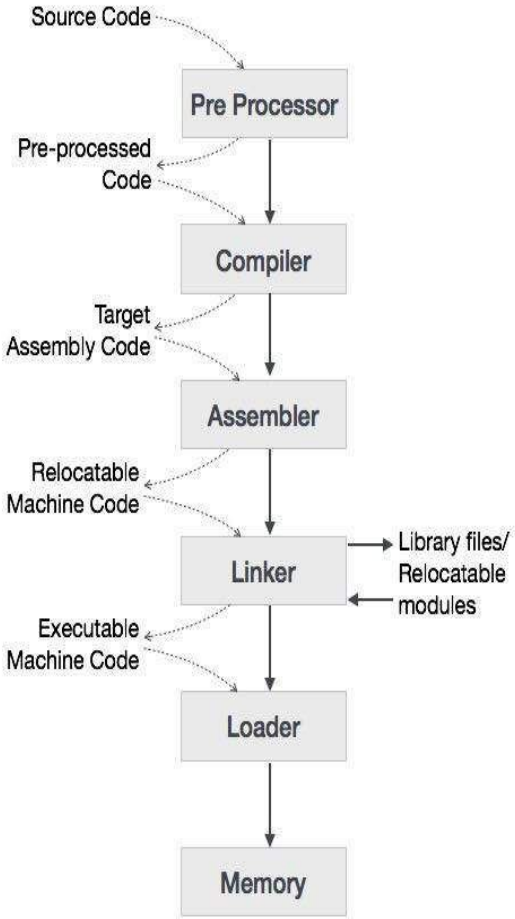


How Compiler Works?

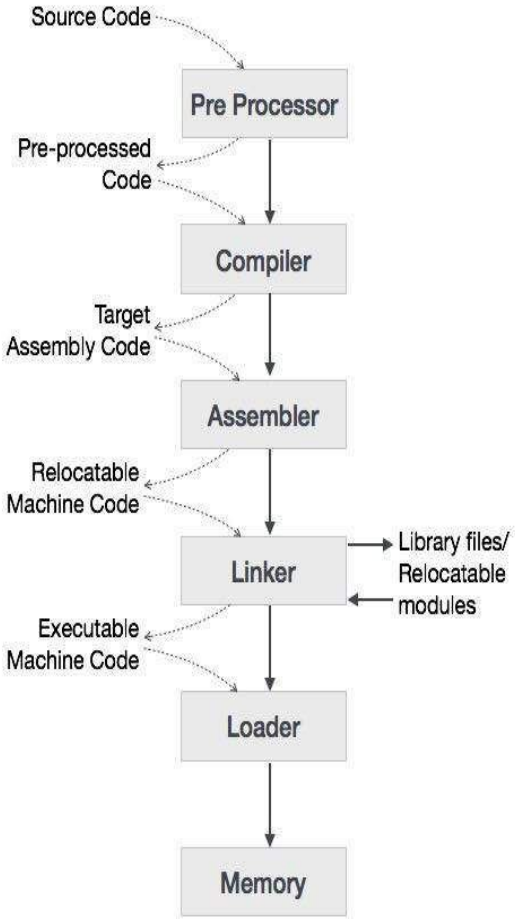
The high-level language is converted into binary language in various phases. A **compiler** is a program that converts high-level language to assembly language. Similarly, an **assembler** is a program that converts the assembly language to machine-level language.

Let us first understand how a program, using C compiler, is executed on a host machine.

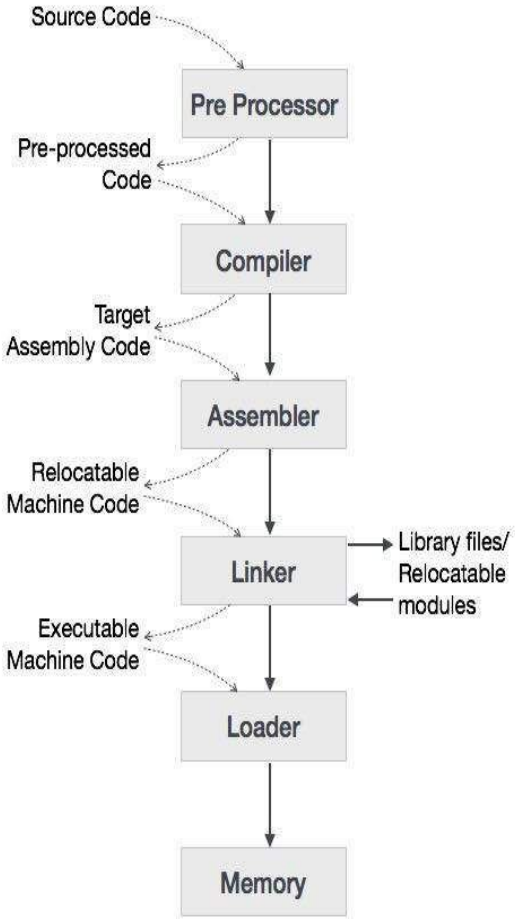
- **User writes a program in C language (high-level language).**
- **The C compiler, compiles the program and translates it to assembly program (low-level language).**
- **An assembler then translates the assembly program into machine code (object).**
- **A linker tool is used to link all the parts of the program together for execution (executable machine code).**
- **A loader loads all of them into memory and then the program is executed.**



- **High-Level Language:** If a program contains pre-processor directives such as #include or #define it is called HLL. They are closer to humans but far from machines. These (#) tags are called preprocessor directives. They direct the pre-processor about what to do.
- **Pre-Processor:** The pre-processor removes all the #include directives by including the files called file inclusion and all the #define directives using macro expansion. It performs file inclusion, augmentation, macro-processing, etc. For example: Let in the source program, it is written #include “Stdio. h”. Pre-Processor replaces this file with its contents in the produced output.
- **Assembly Language:** It’s neither in binary form nor high level. It is an intermediate state that is a combination of machine instructions and some other useful data needed for execution.



- **Assembler:** For every platform (Hardware + OS) we will have an assembler. They are not universal since for each platform we have one. The output of the assembler is called an object file. Its translates assembly language to machine code.
- **Compiler:** The compiler is an intelligent program as compared to an assembler. The compiler verifies all types of limits, ranges, errors, etc. Compiler program takes more time to run and it occupies a huge amount of memory space. The speed of the compiler is slower than other system software. It takes time because it enters through the program and then does the translation of the full program.
- **Interpreter:** An interpreter converts high-level language into low-level machine language, just like a compiler. But they are different in the way they read the input. The Compiler in one go reads the inputs, does the processing, and executes the source code whereas the interpreter does the same line by line. A compiler scans the entire program and translates it as a whole into machine code whereas an interpreter translates the program one statement at a time. Interpreted programs are usually slower concerning compiled ones.



- **Relocatable Machine Code:** It can be loaded at any point and can be run. The address within the program will be in such a way that it will cooperate with the program movement.
- **Loader/Linker:** Loader/Linker converts the relocatable code into absolute code and tries to run the program resulting in a running program or an error message (or sometimes both can happen). Linker loads a variety of object files into a single file to make it executable. Then loader loads it in memory and executes it.
  - **Linker:** The basic work of a linker is to merge object codes (that have not even been connected), produced by the compiler, assembler, standard library function, and operating system resources.
  - **Loader:** The codes generated by the compiler, assembler, and linker are generally re-located by their nature, which means to say, the starting location of these codes is not determined, which means they can be anywhere in the computer memory. Thus the basic task of loaders to find/calculate the exact address of these memory locations.

Types of Compiler

The four main types of compilers are as follows –

1. Single-Pass Compiler – A single-pass compiler processes the source code in a single pass, from start to finish, generating machine code as it goes. It is efficient but may not catch all errors or perform extensive optimization.

2. Multi-Pass Compiler – A multi-pass compiler makes multiple passes over the source code, analyzing it in different stages. This allows for more thorough error checking and optimization but can be slower than a single-pass compiler.

3. Just-In-Time (JIT) Compiler – A JIT compiler translates code into machine language while the program is running, on-the-fly. It is used in languages like Java and JavaScript to improve performance by converting code as needed during execution.

4. Ahead-of-Time (AOT) Compiler – An AOT compiler translates code into machine language before the program is run, producing an executable file. This approach is common in languages like C and C++, providing fast execution but requiring compilation before running the program.

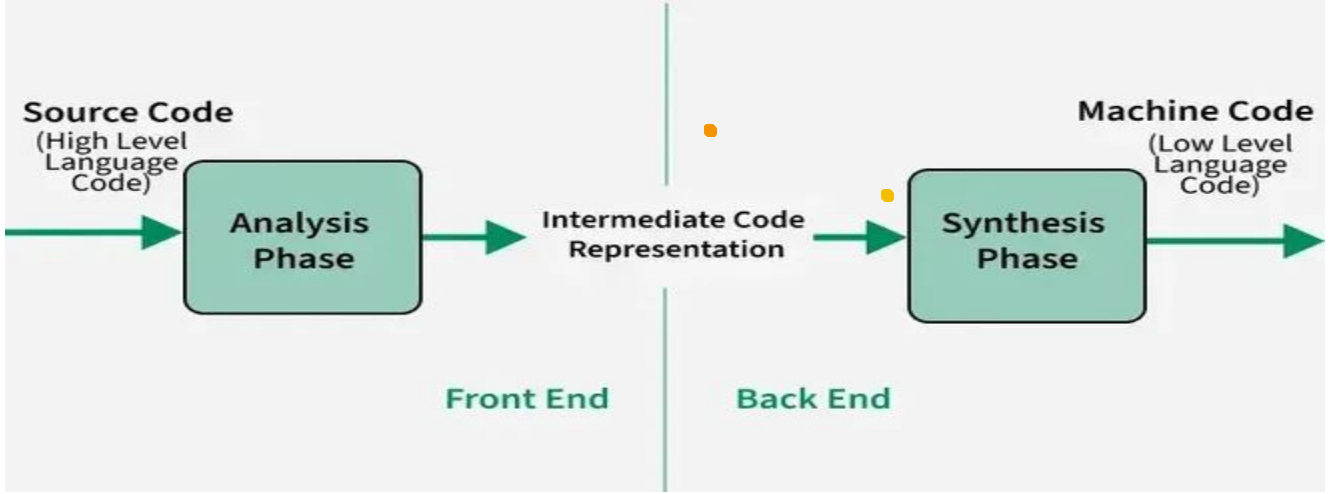
Other types of Compiler

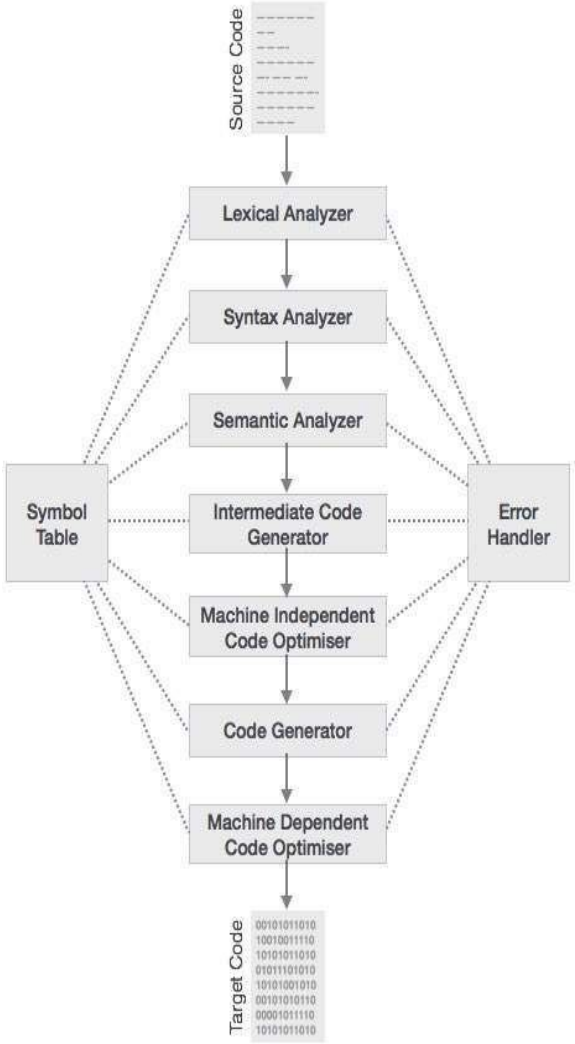
- **Self Compiler:** When the compiler runs on the same machine and produces machine code for the same machine on which it is running then it is called as self compiler or resident compiler.
- **Cross Compiler**: The compiler may run on one machine and produce the machine codes for other computers then in that case it is called a cross-compiler. It is capable of creating code for a platform other than the one on which the compiler is running.
- **Source-to-Source Compiler:** A Source-to-Source Compiler or transcompiler or transpiler is a compiler that translates source code written in one programming language into the source code of another programming language
- **Two Pass Compiler:** Two-pass compiler is a compiler in which the program is translated twice, once from the front end and the back from the back end known as Two Pass Compiler.
- **Incremental Compiler:** It compiles only the parts of the code that have changed, rather than recompiling the entire program. This makes the compilation process faster and more efficient, especially during development.

Structure and Phases of Compiler

A compiler performs this transformation through several phases, each with a specific role in making the code efficient and correct. Broadly, the compilation process can be divided into two main parts:

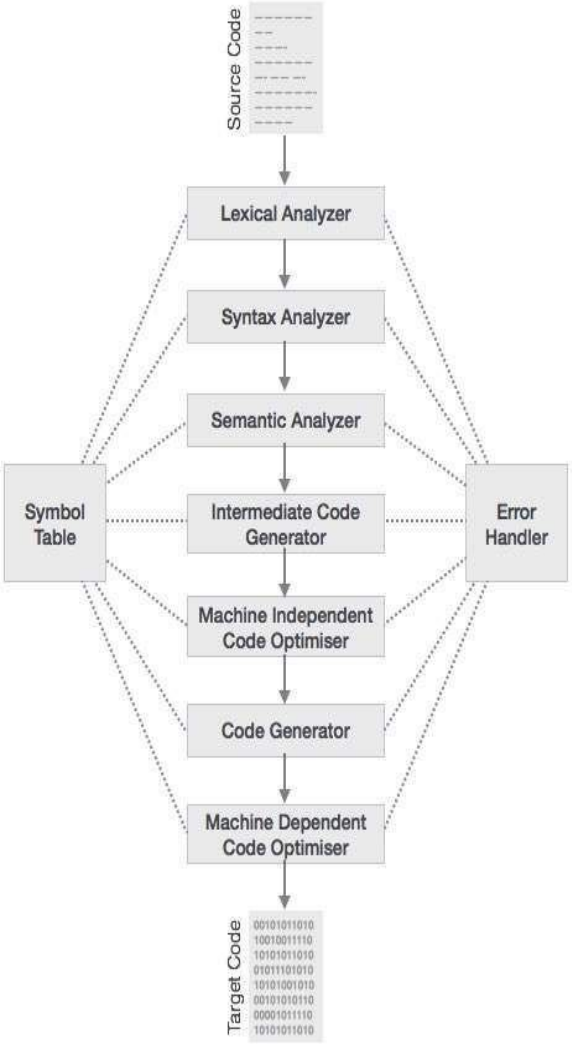
1. **Analysis Phase:** The analysis phase breaks the source program into its basic components and creates an intermediate representation of the program. It is sometimes referred to as front end.
2. **Synthesis Phase:** The synthesis phase creates the final target program from the intermediate representation. It is sometimes referred to as back end.





Phases of a Compiler

- The compiler consists of two main parts: the front-end and the back-end. The front-end includes the **lexical analyzer, syntax analyzer, semantic analyzer, and intermediate code generator**. The back-end takes over from there, **handling optimization, code generation, and assembly.**
- The compilation process is a sequence of various phases. Each phase takes input from its previous stage, has its own representation of source program, and feeds its output to the next phase of the compiler.



1. **Lexical Analysis:** The first phase, where the source code is broken down into tokens such as keywords, operators, and identifiers for easier processing.
2. **Syntax Analysis or Parsing:** This phase checks if the source code follows the correct syntax rules, building a parse tree or abstract syntax tree (AST).
3. **Semantic Analysis:** It ensures the program's logic makes sense, checking for errors like type mismatches or undeclared variables.
4. **Intermediate Code Generation:** In this phase, the compiler converts the source code into an intermediate, machine-independent representation, simplifying optimization and translation.
5. **Code Optimization:** This phase improves the intermediate code to make it run more efficiently, reducing resource usage or increasing speed.
6. **Target Code Generation:** The final phase where the optimized code is translated into the target machine code or assembly language that can be executed on the computer.

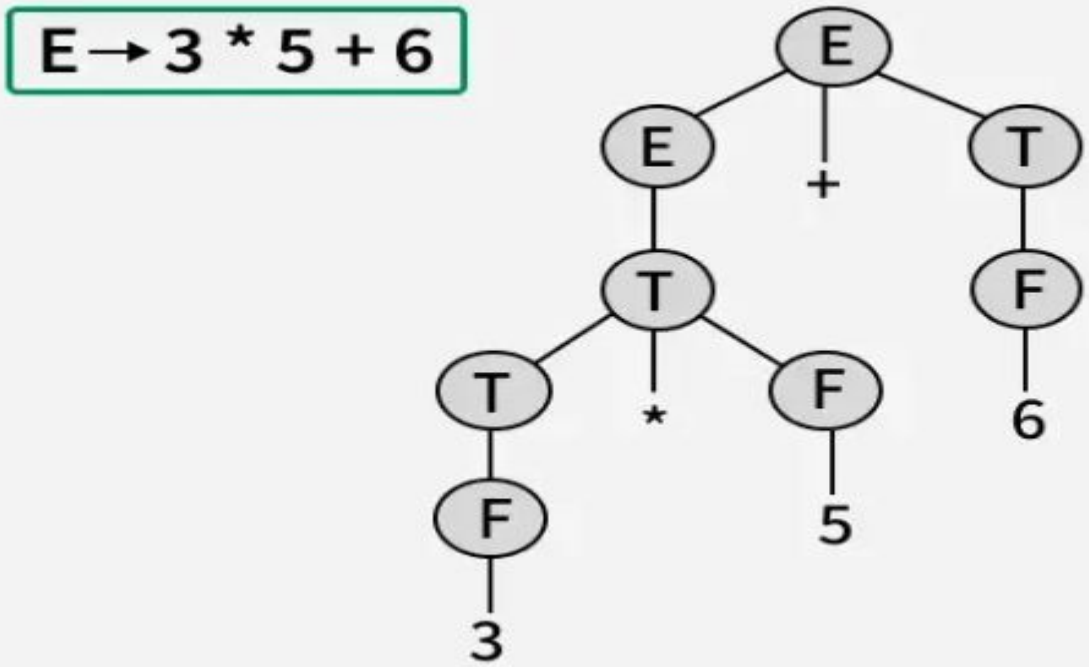
1. Lexical Analysis

Example: int x = 10;

The lexical analyzer would break this line into the following tokens:

int – Keyword token (data type)
x – Identifier token (variable name)
= – Operator token (assignment operator)
10 – Numeric literal token (integer value)
; – Punctuation token (semicolon, used to terminate statements)

2. Syntax Analysis



3. Semantic Analysis

Example:

int a = 5;
float b = 3.5;
a = a + b;

Type Checking:

- `a` *is* `int` *and* `b` *is* `float`*. Adding them (*`a + b`*) results in* `float`*, which cannot be assigned to* `int a`*.*
- ***Error:*** `Type mismatch: cannot assign float to int.`

4. Intermediate Code Generation

*Example: a = b + c * d;*

*t1 = c * d*
t2 = b + t1
a = t2

6. Code Generation

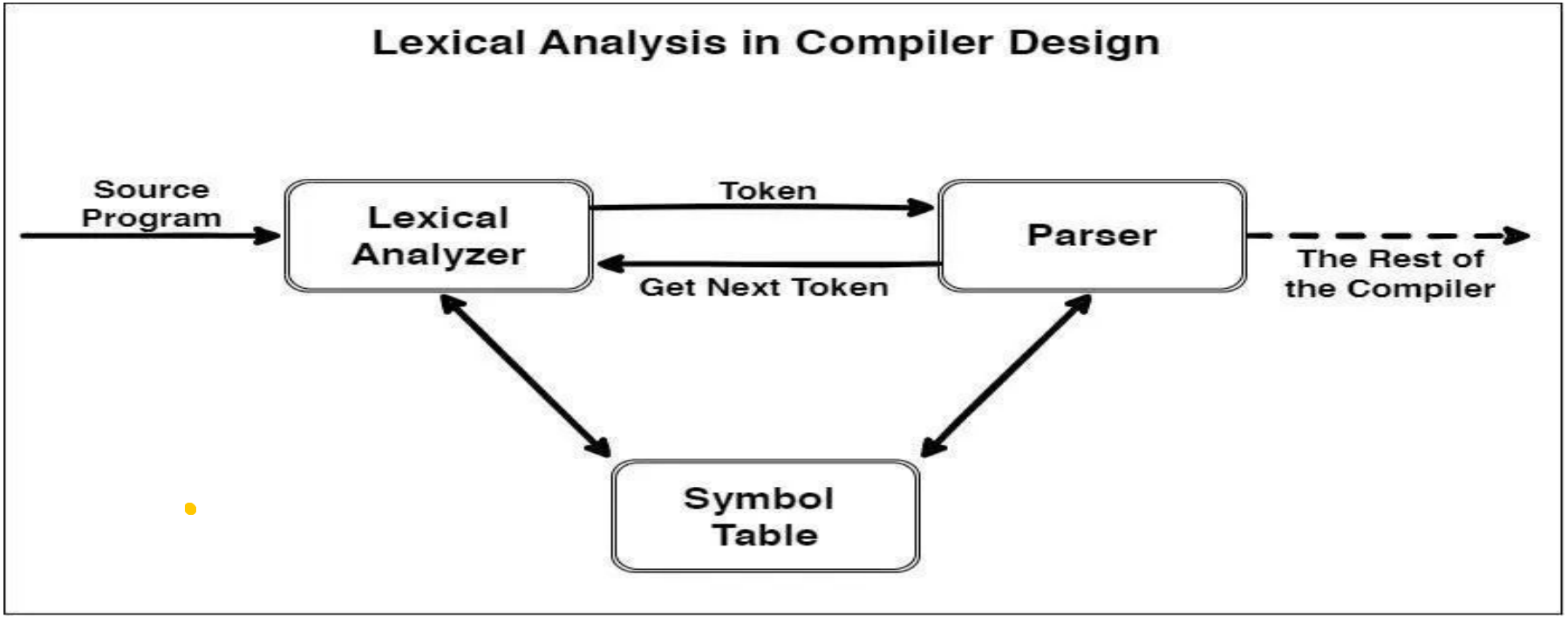
Three Address Code	Assembly Code

t1 = c * d
t2 = b + t1
a = t2
	LOAD R1, c ; Load the value of 'c' into register R1
	LOAD R2, d ; Load the value of 'd' into register R2
	MUL R1, R2 ; R1 = c * d, store result in R1
	LOAD R3, b ; Load the value of 'b' into register R3
	ADD R3, R1 ; R3 = b + (c * d), store result in R3
	STORE a, R3 ; Store the final result in variable 'a'

Symbol Table

It is a data structure being used and maintained by the compiler, consisting of all the identifier's names along with their types. It helps the compiler to function smoothly by finding the identifiers quickly.

Unit 2: Scanner(Lexical Analysis)



Scanner(Lexical Analysis)

Lexical analysis is the first phase of a compiler. It takes modified source code from language preprocessors that are written in the form of sentences. The lexical analyzer breaks these syntaxes into a series of tokens, by removing any whitespace or comments in the source code.

Tokens are meaningful sequences of characters. There are usually only a small number of tokens for a programming language including constants (such as integers, doubles, characters, and strings), operators (arithmetic, relational, and logical), punctuation marks and reserved keywords.

What is a Token?
A lexical token is a sequence of characters that can be treated as a unit in the grammar of the programming languages.

Tokens

In programming language, keywords, constants, identifiers, strings, numbers, operators and punctuations symbols can be considered as tokens.

For example, in C language, the variable declaration line

```
int value = 100;
```

contains the tokens:

```
int (keyword), value (identifier), = (operator), 100 (constant) and ;
(symbol).
```

2.1 Role of Scanner

In compiler design, the scanner (also called a lexical analyzer or tokenizer) functions to identify meaningful sequences of characters in the source code and convert them into tokens. These tokens are then passed to the parser for further processing.

Here's a more detailed breakdown of the scanner's functions:

1. Input Processing:

- The scanner takes the source code as a stream of characters.
- It reads this stream character by character.
- It may use input buffering to read larger chunks of input at a time.

2. Tokenization:

- The scanner identifies and extracts meaningful units called tokens.
- Examples of tokens include keywords, identifiers, operators, literals, and punctuation.
- Tokens can be described using regular expressions.
- The scanner uses a deterministic finite automaton (DFA) to recognize these tokens.

3. Token Classification:

- The scanner assigns a type (e.g., keyword, identifier, operator) to each recognized token.
- It may also store information about the token, such as its line number or the text it represents.

4. Error Handling:

- The scanner detects and reports errors in the input, such as invalid characters or unexpected symbols.
- It might generate an error message or code to indicate the type of error.

5. Token Output:

- The scanner produces a stream of tokens, which are then passed to the parser.
- The parser uses these tokens to build a parse tree and eventually generate intermediate code.

2.2 Recognizing Words

In lexical analysis (also called lexing or scanning), recognizing words means identifying meaningful sequences of characters from the input source code—these sequences are called lexemes, and they are matched against patterns defined for tokens.

What is a “word”?

A "word" is usually a token—a basic building block of the source code. Examples of words include:

1. Keywords

Reserved words in the programming language.

- Examples: `if`, `else`, `while`, `return`, `int`, `for`, `class`

2. Identifiers

Names defined by the programmer.

- Examples: `sum`, `totalMarks`, `myFunction`, `x`, `i`
- Rules vary by language (e.g., must start with a letter or underscore)

3. Literals (Constants)

Fixed values written directly in the code.

- **Integer literals**: `10`, `0`, `-99`
- **Float literals**: `3.14`, `0.0`, `-2.5`
- **Character literals**: `'a'`, `'1'`, `'\n'`
- **String literals**: `"hello"`, `"Anil"`

4. Operators

Symbols that perform operations on operands.

- **Arithmetic**: `+`, `-`, `*`, `/`, `%`
- **Relational**: `==`, `!=`, `<`, `>`, `<=`, `>=`
- **Logical**: `&&`, `||`, `!`
- **Assignment**: `=`, `+=`, `-=`

5. Separators / Delimiters

Symbols that separate code elements.

Examples: `(`, `)`, `{`, `}`, `[`, `]`, `,`, `;`, `:`

MAY OR MAYNOT BE A TOKEN

6. Comments (sometimes treated as tokens)

Used for code documentation; ignored during execution.

- Examples: `// this is a comment`, `/* block comment */`

7. Whitespace and Newlines

Usually ignored but can be important in some languages (like Python).

- **Whitespace**: spaces, tabs
- **Newlines**: `\n`

How are words recognized?

1. **Regular expressions** are used to define patterns for each type of token.
2. The lexer (or scanner) scans the input character by character.
3. When it finds a sequence that matches a pattern, it groups those characters into a **token** and classifies it.

Example: For this code:

```
int x = 10;
```

The lexer might recognize the following tokens:

- `int` → **Keyword**
- `x` → **Identifier**
- `=` → **Assignment Operator**
- `10` → **Integer Literal**
- `;` → **Semicolon**

What are "complex words"?

“complex words” typically refer to:

- Multi-character identifiers: `userName123`
- Compound operators: `+=`, `==`, `&&`
- Floating-point numbers: `3.14159`, `0.001`
- Strings with escape sequences: `"Hello\nWorld"`
- Comments (which might span lines or contain symbols)
- Keywords vs. identifiers (e.g., `int` is a keyword, `intVar` is an identifier)

Regular Expression

The lexical analyzer needs to scan and identify only a finite set of valid string/token/lexeme that belong to the language in hand. It searches for the pattern defined by the language rules.

Regular expressions have the capability to express finite languages by defining a pattern for finite strings of symbols. The grammar defined by regular expressions is known as **regular grammar**. The language defined by regular grammar is known as **regular language**.

Regular expression is an important notation for specifying patterns. Each pattern matches a set of strings, so regular expressions serve as names for a set of strings. Programming language tokens can be described by regular languages. The specification of regular expressions is an example of a recursive definition. Regular languages are easy to understand and have efficient implementation.

Difference between Null Strings and Empty Sets

A **null string** () is a valid string with no characters. It is like a blank piece of paper. For example, consider the language {, 0, 00}. It includes the null string and strings of zeros, but it is not an empty set because it contains elements.

An **empty set** (φ), on the other hand, contains no strings at all. It is like having no paper to write on.

Closure properties of Regular languages

Closure properties on regular languages are defined as certain operations on regular language that are guaranteed to produce regular language. Closure refers to some operation on a language, resulting in a new language that is of the same “type” as originally operated on i.e., regular.

Operation	Description
Union	Combining two languages, L1 and L2, by taking all strings that are in either L1 or L2.
Concatenation	Combining two languages, L1 and L2, by creating all strings where a string from L1 is followed by a string from L2.
Closure (Kleene Star)	Creating a new language by taking all possible strings formed by concatenating zero or more copies of strings from the original language.
Complement	Creating a new language by taking all strings over the alphabet that are not present in the original language.
Intersection	Creating a new language by taking all strings that are present in both L1 and L2.
Difference	Creating a new language by taking all strings that are in L1 but not in L2.
Reversal	Creating a new language by reversing each string in the original language.
Homomorphism	Replacing each symbol in a language with another symbol according to a mapping rule.

Reverse Homomorphism	The reverse operation of homomorphism.
Quotient	Creating a new language by dividing one language by another.
Initiate	Creating a new language by taking all prefixes of strings in the original language.
Substitution	Replacing each symbol in a language with another language according to a mapping rule.
Infinite Union	Creating a new language by combining an infinite number of languages.

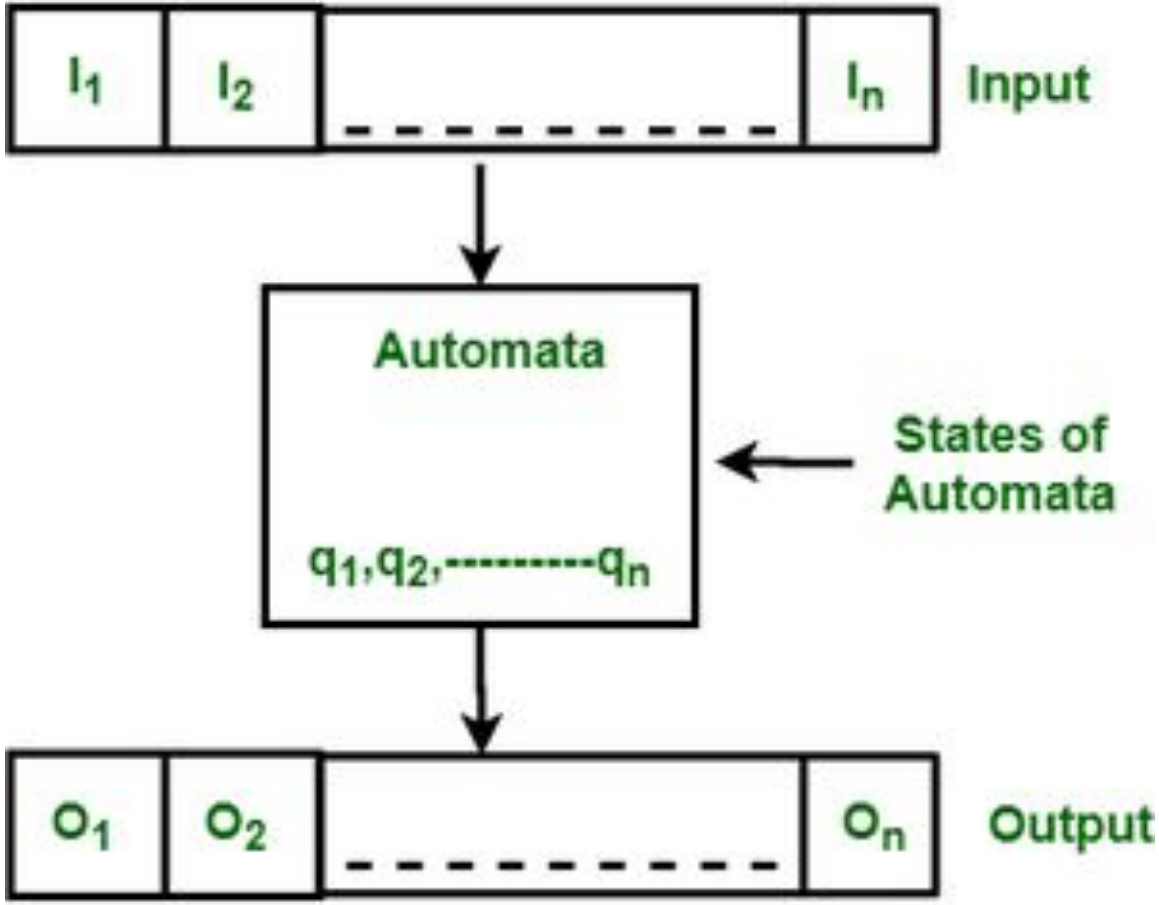
Finite Automata

Finite automata is a state machine that takes a string of symbols as input and changes its state accordingly.

Finite automata is a recognizer for regular expressions. When a regular expression string is fed into finite automata, it changes its state for each literal.

If the input string is successfully processed and the automata reaches its final state, it is accepted, i.e., the string just fed was said to be a valid token of the language in hand.

Finite automata are abstract machines used to recognize patterns in input sequences, forming the basis for understanding regular languages in computer science. They consist of states, transitions, and input symbols, processing each symbol step-by-step. If the machine ends in an accepting state after processing the input, it is accepted; otherwise, it is rejected.


I_1
I_2
I_n
Input
Automata
$q_1, q_2, \text{---------} q_n$
States of Automata
O_1
O_2
O_n
Output

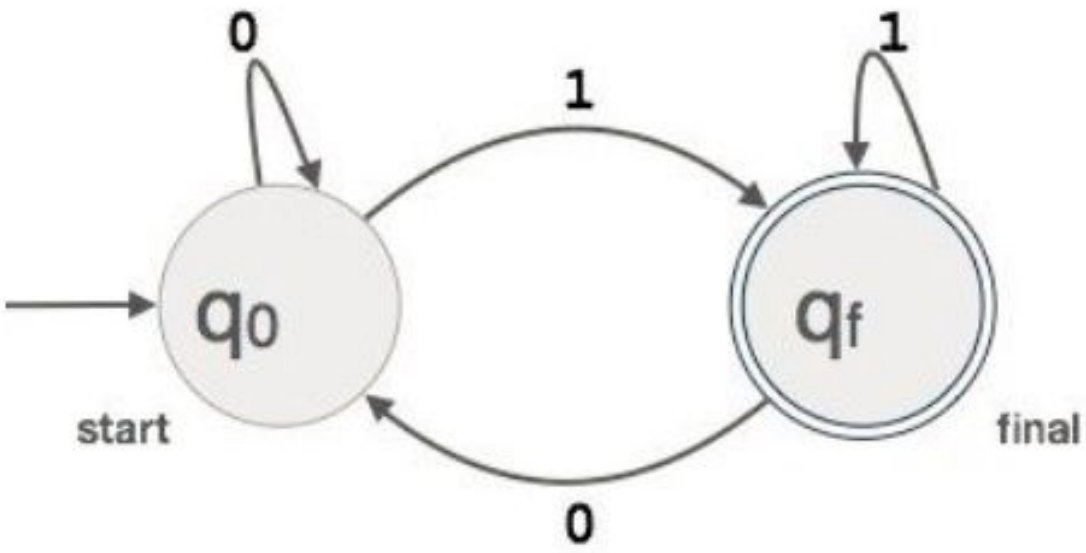
Mathematical Model of Finite Automata

A finite automaton can be defined as a tuple:

$\{ Q, \sum, q, F, \delta \}$, where:

- Q: Finite set of states
- $\sum$: Set of input symbols
- q: Initial state
- F: Set of final states
- δ: Transition function

Example : We assume FA accepts any three digit binary value ending in digit 1.

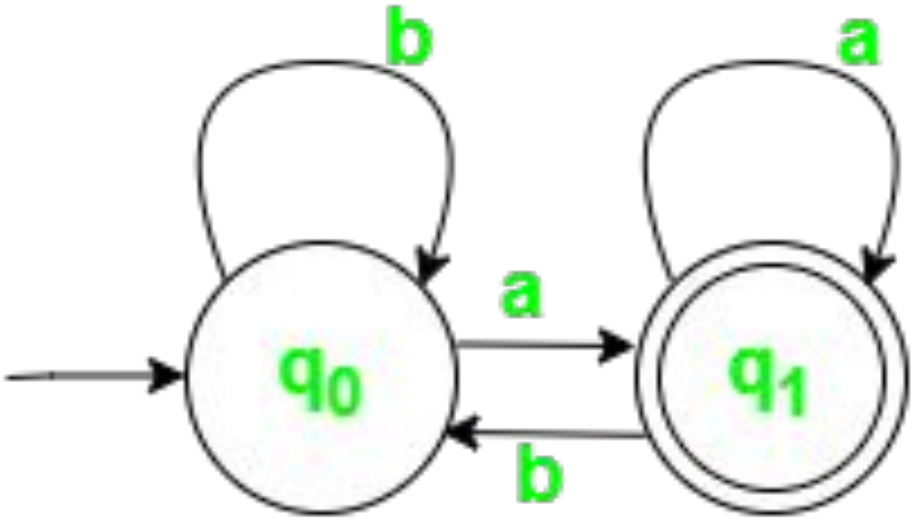


1. Deterministic Finite Automata (DFA)

A DFA is represented as {Q, ∑, q, F, δ}. In DFA, for each input symbol, the machine transitions to one and only one state. DFA does not allow any null transitions, meaning every state must have a transition defined for every input symbol.

```
DFA consists of 5 tuples {Q, Σ, q, F, δ}.

Q : set of all states.

Σ : set of input symbols. ( Symbols which machine takes as input )

q : Initial state. ( Starting state of a machine )

F : set of final state.

δ : Transition Function, defined as δ : Q X Σ --> Q.
```

Construct a DFA that accepts all strings ending with ‘a’.



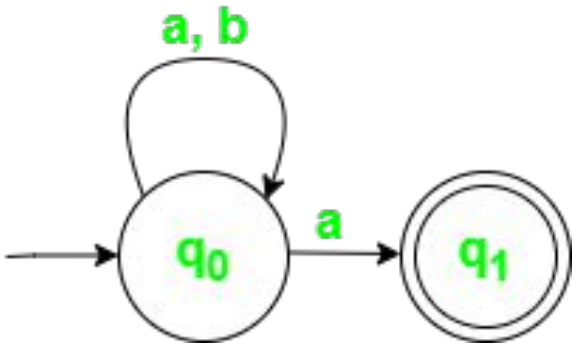
State\Symbol	a	b
q_0	q_1	q_0
q_1	q_1	q_0

2) Non-Deterministic Finite Automata (NFA)

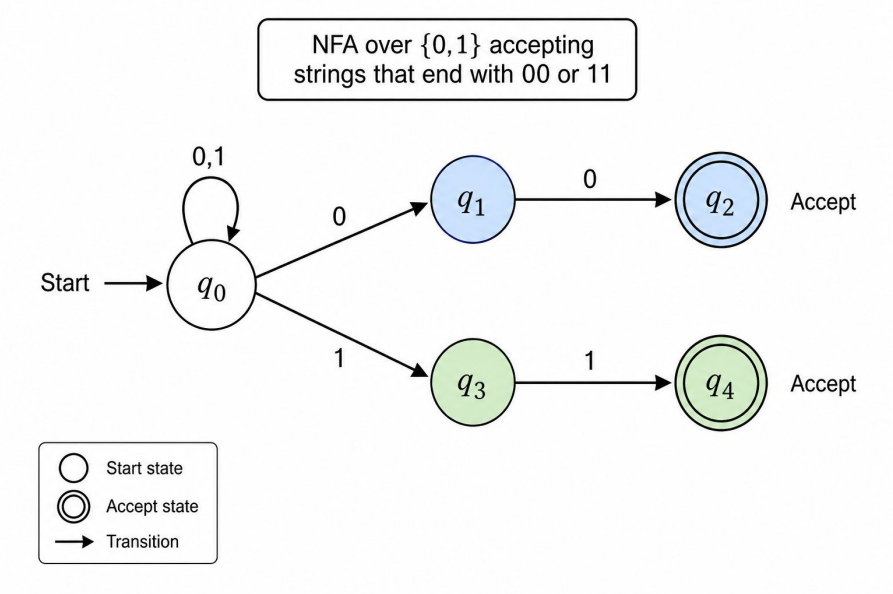
NFA is similar to DFA but includes the following features:

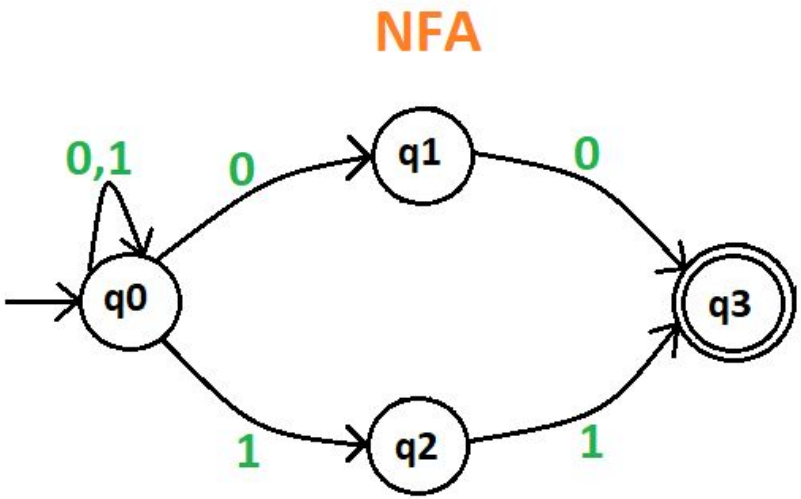
- It can transition to multiple states for the same input.
- It allows null (ϵ) moves, where the machine can change states without consuming any input.

State\Symbol	a	b
q_0	$\{q_0, q_1\}$	q_0
q_1	φ	φ

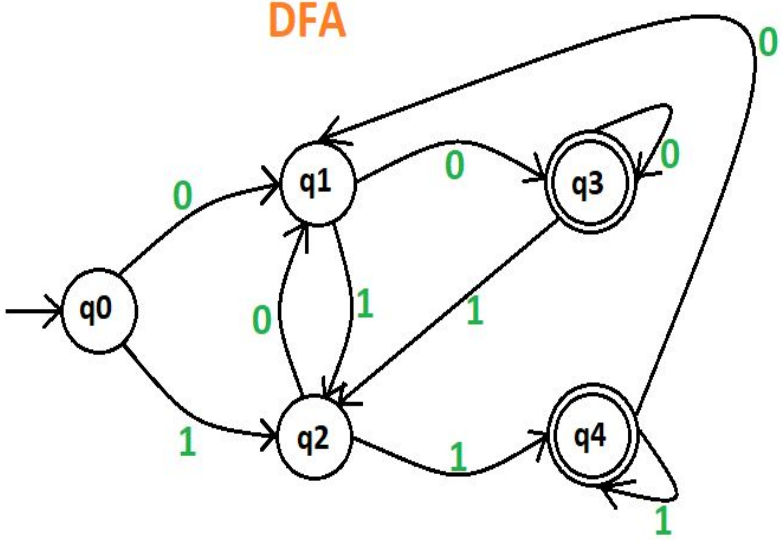


Draw a non deterministic finite automate which accept 00 and 11 at the end of a string containing 0, 1 in it, e.g., 01010100 but not 000111010





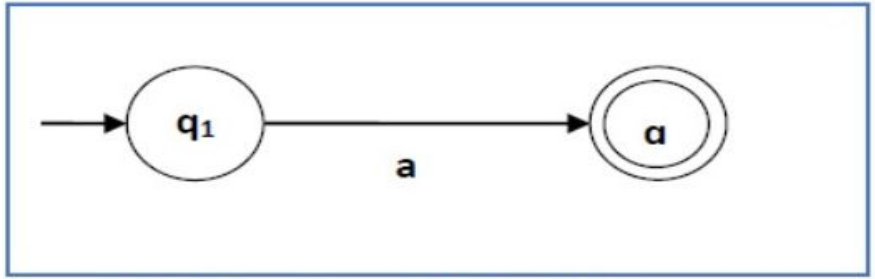
DFA



Regular Expression to NFA: Thomson's Construction

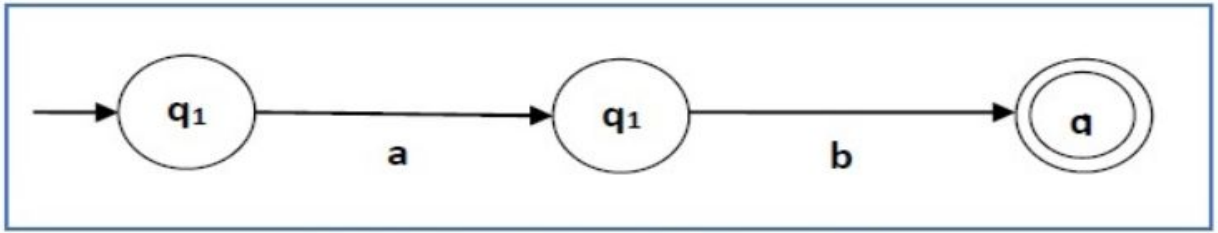
We can use Thompson's Construction to find out a Finite Automaton from a Regular Expression. We will reduce the regular expression into smallest regular expressions and converting these to NFA and finally to DFA.

Case 1 – For a regular expression a, we can construct the following FA –



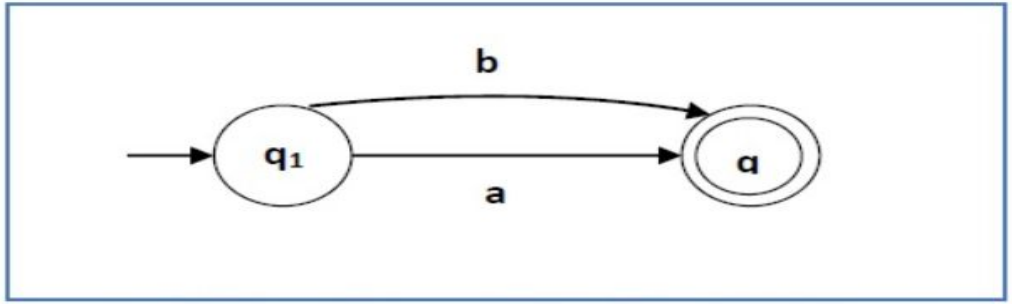
Finite automata for RE = a

Case 2 – For a regular expression ab, we can construct the following FA –



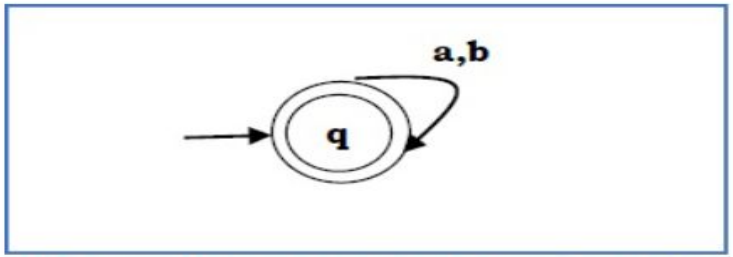
Finite automata for RE = ab

Case 3 – For a regular expression (a+b), we can construct the following FA –



Finite automata for RE= (a+b)

Case 4 – For a regular expression (a+b)*, we can construct the following FA –



Finite automata for RE= (a+b)*

Unit 3: Parser

3.1 Introduction to syntax analyzer.

3.2 Context Free Grammar.

3.3 Top Down Parsing

- 3.3.1 Transforming a grammar for top down parsing
- 3.3.2 Recursive Descent Parsing
- 3.3.3 Table driver LL(1) Parser

3.4 Bottom Up parsing

- 3.4.1 The LR(1) Parsing Algorithm
- 3.4.2 Building LR(1) Table

Syntax Analyzer OR Parser

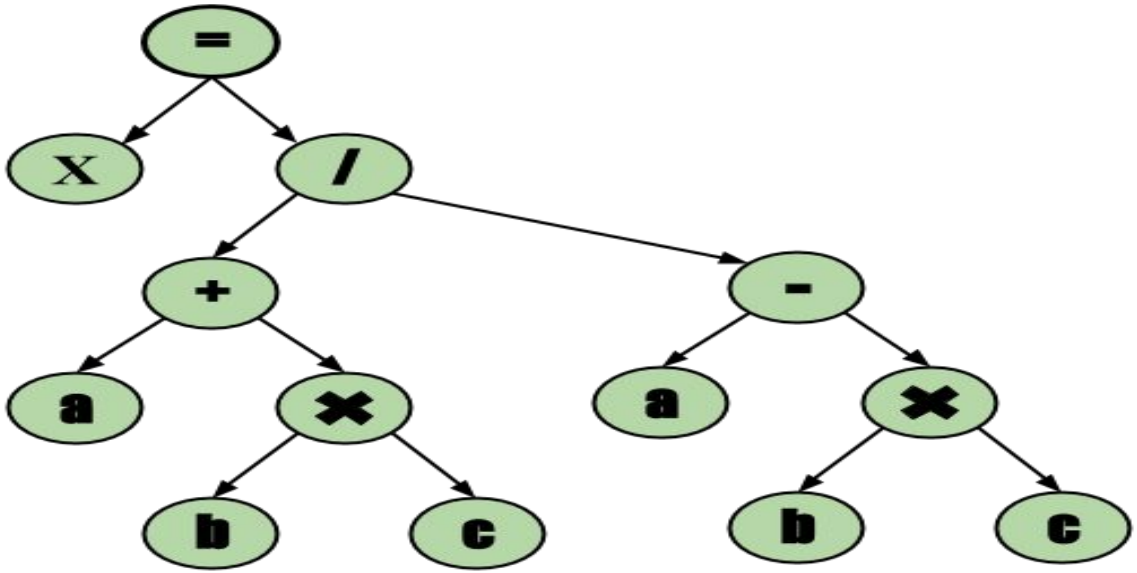
Parsing, also known as syntactic analysis, is the process of analyzing a sequence of tokens to determine the grammatical structure of a program. It takes the stream of tokens, which are generated by a lexical analyzer or tokenizer, and organizes them into a parse tree or syntax tree.

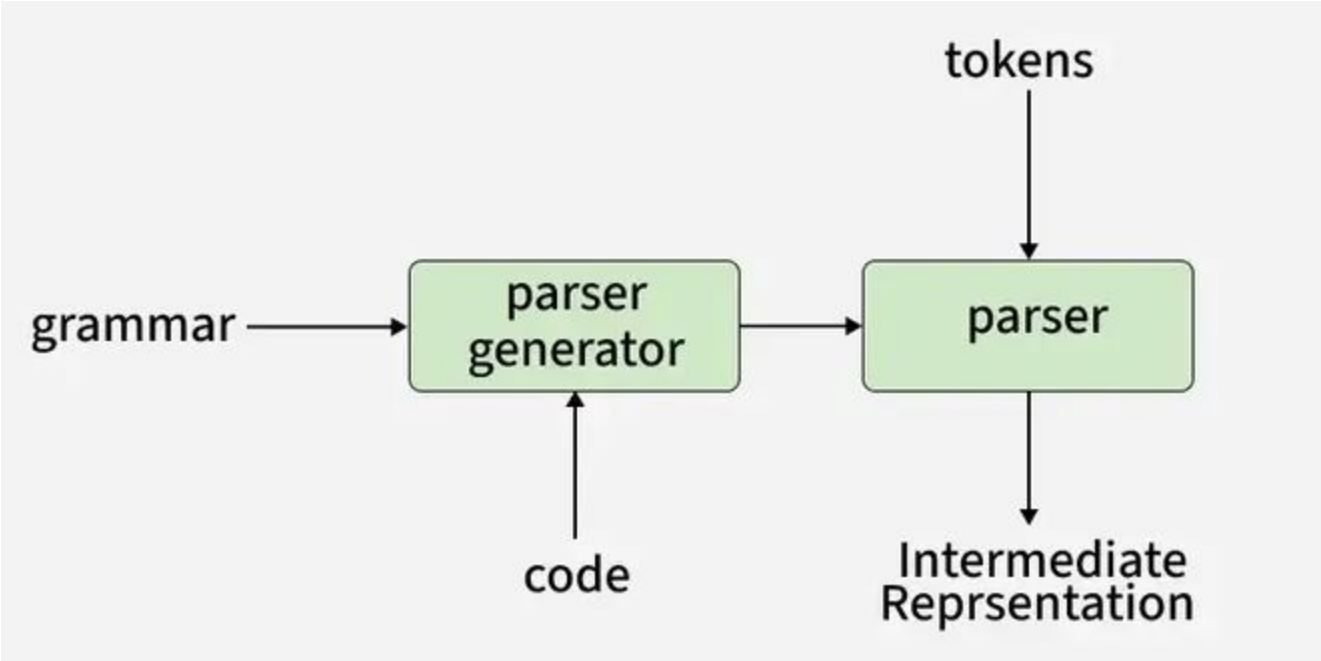
We have seen that a lexical analyzer can identify tokens with the help of regular expressions and pattern rules. But a lexical analyzer cannot check the syntax of a given sentence due to the limitations of the regular expressions. Regular expressions cannot check balancing tokens, such as parenthesis. Therefore, this phase uses context-free grammar (CFG), which is recognized by push-down automata.

The parse tree visually represents how the tokens fit together according to the rules of the language's syntax. This tree structure is crucial for understanding the program's structure and helps in the next stages of processing, such as code generation or execution.

X = (a + (b* c)) / (a - (b * c))

Operator Root




tokens
grammar
parser generator
parser
code
Intermediate Reprsentation

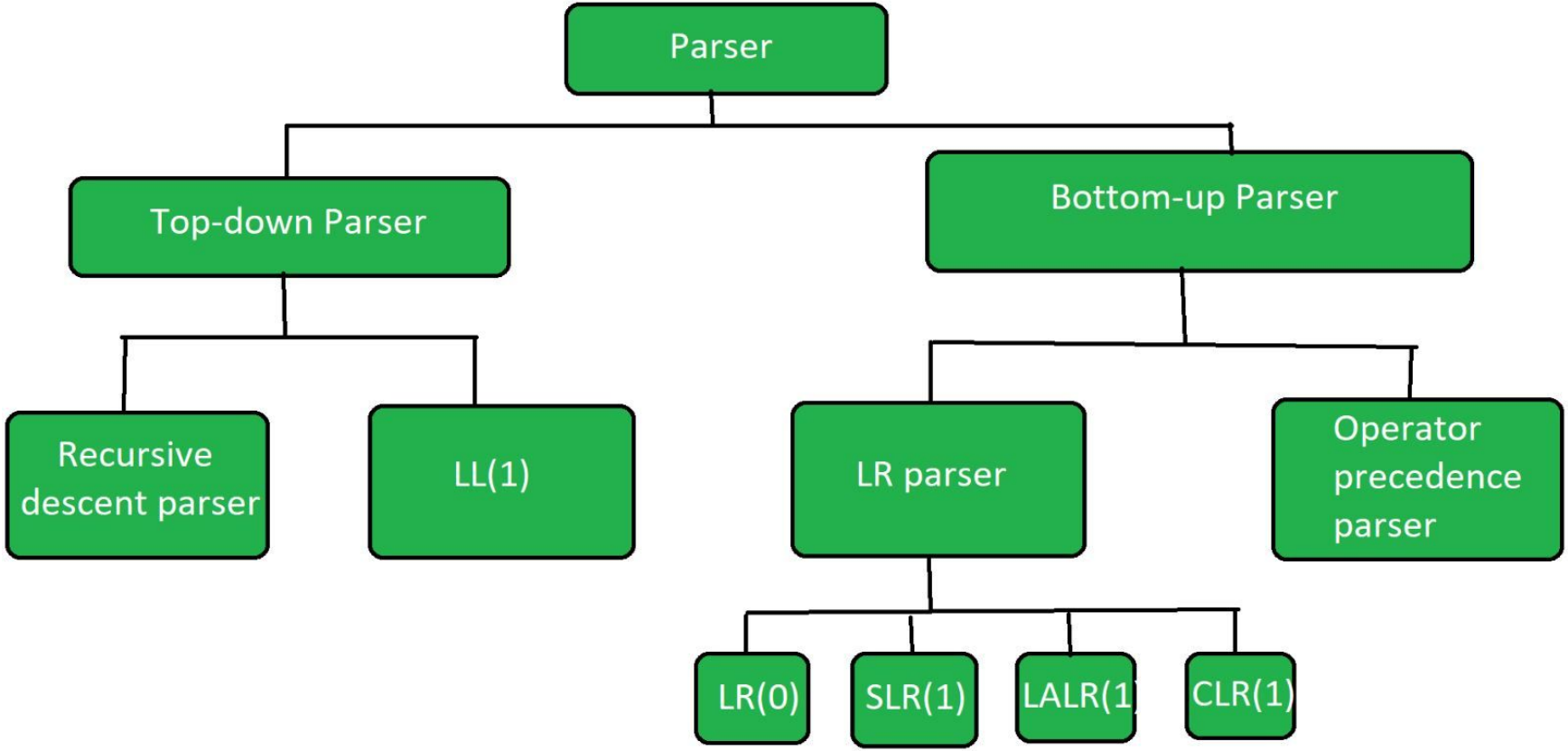
What is the Role of Parser?

A parser performs syntactic and semantic analysis of source code, converting it into an intermediate representation while detecting and handling errors.

1. **Context-free syntax analysis**: The parser checks if the structure of the code follows the basic rules of the programming language (like grammar rules). It looks at how words and symbols are arranged.
2. **Guides context-sensitive analysis**: It helps with deeper checks that depend on the meaning of the code, like making sure variables are used correctly. For example, it ensures that a variable used in a mathematical operation, like x + 2, is a number and not text.
3. **Constructs an intermediate representation**: The parser creates a simpler version of your code that's easier for the computer to understand and work with.
4. **Produces meaningful error messages**: If there's something wrong in your code, the parser tries to explain the problem clearly so you can fix it.
5. **Attempts error correction**: Sometimes, the parser tries to fix small mistakes in your code so it can keep working without breaking completely.

Context-Free Grammar

Types of Parsers



Transforming a grammar for top down parsing

To make a grammar suitable for top-down parsing, it needs to be transformed to remove left recursion and left-factoring.

These transformations ensure that the grammar can be parsed efficiently using a top-down parser like an LL(1) parser, which reads the input from left to right and constructs a left-most derivation.

1. Eliminating Left Recursion:

What is left recursion?

A grammar is left-recursive if a non-terminal can directly or indirectly derive a string that starts with itself.

Why is it problematic for top-down parsing?

Left-recursive grammars can lead to infinite loops in top-down parsers because the parser might repeatedly expand the same non-terminal without making progress.

How to eliminate it?

- **Direct left recursion:** If you have A -> A α | β, where α and β don't start with A, transform it into:
  - `A -> β A' `
  - A' -> α A' | ε (ε is the empty string)
- **Indirect left recursion:** If you have A -> B α and B -> A β, you'll need to eliminate the recursion through the B non-terminal first, and then apply the direct left recursion elimination to A.

2. Left Factoring:

What is left-factoring?

A grammar is left-factored if it has two or more productions with the same non-terminal on the left side that share a common prefix.

Why is it problematic for top-down parsing?

Left-factoring can make it difficult for a top-down parser to choose the correct production when it encounters a shared prefix in the input.

How to eliminate it?

- **Identify shared prefixes:** Find productions with the same non-terminal that start with the same symbols.
- **Introduce a new non-terminal:** Factor out the common prefix and introduce a new non-terminal (e.g., A') to represent the rest of the production.
  - For example, if you have `A -> α β | α γ`, you can transform it to `A -> α A' ` and `A' -> β | γ`.

Top-Down Parser

Top-down parser is the parser that generates parse tree for the given input string with the help of grammar productions by expanding the non-terminals. It starts from the start symbol and ends down on the terminals. It uses left most derivation.

Further Top-down parser is classified into 2 types:

1. **Recursive descent parser** is also known as the Brute force parser or the backtracking parser. It basically generates the parse tree by using brute force and backtracking techniques.
2. **Non-recursive descent parser is also known as LL(1)** parser or predictive parser or without backtracking parser or dynamic parser. It uses a parsing table to generate the parse tree instead of backtracking.

Recursive Descent Parser

A **recursive descent parser** is a top-down parser that processes input based on a set of recursive functions, where each function corresponds to a grammar rule. It parses the input from left to right, constructing a parse tree by matching the grammar's production rules.

Algorithm for Recursive Descent Parser

```
S()
{     Choose any S production, S ->X1X2…..Xk;
      for (i = 1 to k)
      {
          If ( Xi is a non-terminal)
          Call procedure Xi();
          else if ( Xi equals the current input, increment input)
          Else /* error has occurred, backtrack and try another possibility */
      }
}
```

The given grammar is:

```
E  → i E'
E' → + i E' | ε
```

Function `E()`

```
E()
{
    if (input == 'i') {  // If the input is 'i' (identifier)
    input++;        // Consume 'i'
}
   E'();     // Call E' to check for further expressions
}
```

- It checks for `i` (identifier).
- If found, it moves the input pointer ahead.
- Calls `E'()` to check if a `+` operation exists.

Function `E'()`

```
void E`() {
    if (input == '+') {
        input++;            // Consume the '+'

        if (input == 'i') {
            input++;        // Consume the 'i'
        }

        E`();            // Recursively process more additions
    } else {
        return;             // If no '+', return (ε production)
    }
}
```

- It checks for `+ i`.
- If found, it consumes them and calls `E'()` recursively.
- If no `+`, it returns (ε production).

Main Function

```
Main()
{
    E();              // Start parsing from E
    if (input == '$') // If we reach end of input
    Parsing Successful;
}
```

- Calls `E()` to start parsing.
- Checks if the input ends with `$`, which indicates a successful parse.

What is LL(1) Parsing?

Here the 1st **L** represents that the scanning of the Input will be done from the Left to Right manner and the second **L** shows that in this parsing technique, we are going to use the Left most Derivation Tree. And finally, the **1** represents the number of look-ahead, which means how many symbols you will see when you want to make a decision.

Conditions for an LL(1) Grammar

To construct a working LL(1) parsing table, a grammar must satisfy these conditions:

- No Left Recursion: Avoid recursive definitions like A -> A + b.
- Unambiguous Grammar: Ensure each string can be derived in only one way.
- Left Factoring: Make the grammar deterministic, so the parser can proceed without guessing.

Algorithm to Construct LL(1) Parsing Table

Step 1: First check all the essential conditions mentioned above and go to step 2.

Step 2: Calculate First() and Follow() for all non-terminals.

1. **First():** If there is a variable, and from that variable, if we try to drive all the strings then the beginning Terminal Symbol is called the First.
2. Follow(): What is the Terminal Symbol which follows a variable in the process of derivation.

Step 3: For each production A –> α. (A tends to alpha)

1. Find First(α) and for each terminal in First(α), make entry A –> α in the table.
2. If First(α) contains ε (epsilon) as terminal, then find the Follow(A) and for each terminal in Follow(A), make entry A –> ε in the table.
3. If the First(α) contains ε and Follow(A) contains $ as terminal, then make entry A –> ε in the table for the $.

To construct the parsing table, we have two functions:

In the table, rows will contain the Non-Terminals and the column will contain the Terminal Symbols. All the **Null Productions** of the Grammars will go under the Follow elements and the remaining productions will lie under the elements of the First set.

Example 1: Consider the Grammar:

```
E --> TE'
E' --> +TE' | ε
T --> FT'
T' --> *FT' | ε
F --> id | (E)

*ε denotes epsilon
```

Step 1: The grammar satisfies all properties

Step 2: Calculate first() and follow().

Find their First and Follow sets:

	First	Follow
E –> TE'	{ id, (}	{ $,) }
E' –> +TE'/ ε	{ +, ε }	{ $,) }
T –> FT'	{ id, (}	{ +, $,) }
T' –> *FT'/ ε	{ *, ε }	{ +, $,) }
F –> id/(E)	{ id, (}	{ *, +, $,) }

Step 3: Make a parser table:

Now, the LL(1) Parsing Table is:

	id	+	*	(	)	$
E	E –> TE’			E –> TE’		
E’		E’ –> +TE’			E’ –> ε	E’ –> ε
T	T –> FT’			T –> FT’		
T’		T’ –> ε	T’ –> *FT’		T’ –> ε	T’ –> ε
F	F –> id			F –> (E)		

Bottom-Up Parser

Bottom-up parsing starts from the leaf nodes of a tree and works in upward direction till it reaches the root node. Here, we start from a sentence and then apply production rules in reverse manner in order to reach the start symbol.

Shift-Reduce Parsing

Shift-reduce parsing uses two unique steps for bottom-up parsing. These steps are known as shift-step and reduce-step.

Shift step: The shift step refers to the advancement of the input pointer to the next input symbol, which is called the shifted symbol. This symbol is pushed onto the stack. The shifted symbol is treated as a single node of the parse tree.

Reduce step : When the parser finds a complete grammar rule (RHS) and replaces it to (LHS), it is known as reduce-step. This occurs when the top of the stack contains a handle. To reduce, a POP function is performed on the stack which pops off the handle and replaces it with LHS non-terminal symbol.

LR Parser

The LR parser is a non-recursive, shift-reduce, bottom-up parser. It uses a wide class of context-free grammar which makes it the most efficient syntax analysis technique. LR parsers are also known as LR(k) parsers, where L stands for left-to-right scanning of the input stream; R stands for the construction of right-most derivation in reverse, and k denotes the number of lookahead symbols to make decisions.

LR(1) LR Parser:

a. Works on complete set of LR(1) Grammar
b. Generates large table and large number of states
c. Slow construction

LR Parsing Algorithm

```
token = next_token()

repeat forever
   s = top of stack
   if action[s, token] = shift si then
      PUSH token
      PUSH si
      token = next_token()
   else if action[s, token] = reduce A::=  then
      POP 2 * || symbols
      s = top of stack
      PUSH A
      PUSH goto[s,A]

   else if action[s, token] = accept then
      return
   else
      error()
```

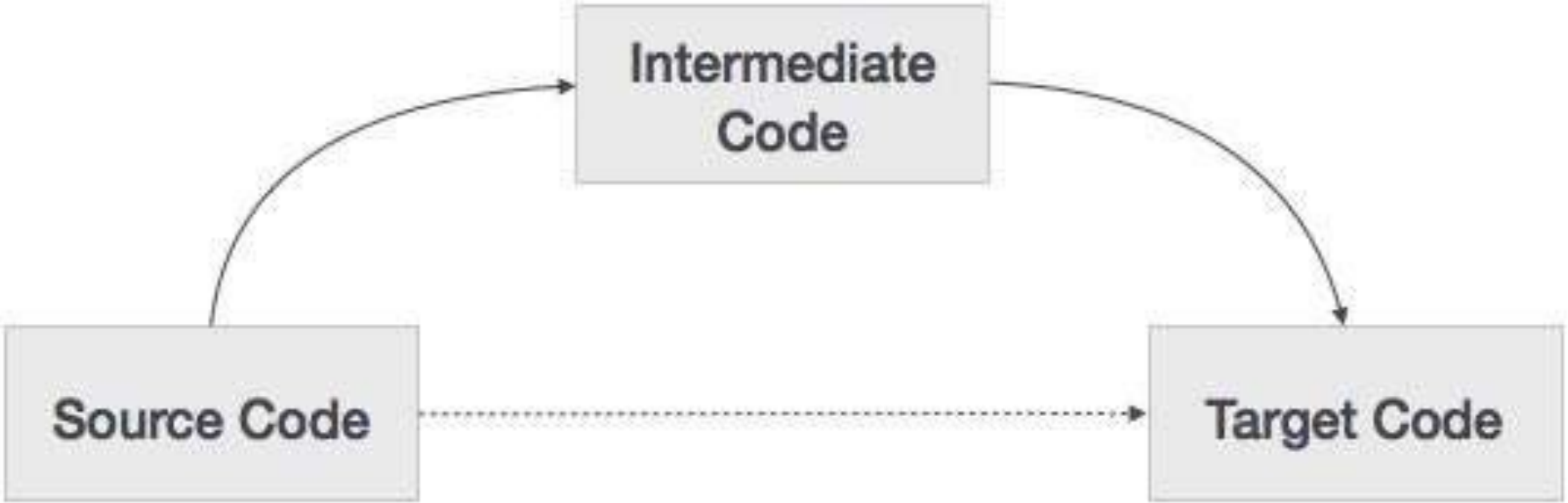
Unit 5: Intermediate Representation(IR)

Intermediate Representation(IR), as the name suggests, is any representation of a program between the source and target languages.

The intermediate form of the program that is being compiled is the ***central data structure*** in a compiler. A compiler may have a single IR or a series of IRs. The decisions that are made during the design of IR affect the efficiency and speed of the compiler.

The below five are the properties of IRs:

1. Ease of generation
2. Ease of manipulation
3. Freedom of expression
4. Size of the procedure
5. Level of abstraction


Intermediate Code
Source Code
Target Code

If a compiler translates the source language to its target machine language without having the option for generating intermediate code, then for each new machine, a full native compiler is required.

Intermediate code eliminates the need of a new full compiler for every unique machine by keeping the analysis portion same for all the compilers.

The second part of compiler, synthesis, is changed according to the target machine.

It becomes easier to apply the source code modifications to improve code performance by applying code optimization techniques on the intermediate code.

Reasons for using Intermediate Representations(IRs):

1. Translating the given code from one form to another requires synthesis and analysis.
2. To perform machine-independent optimizations.
3. To make translation simpler.

- **Translation:** The compiler takes the high-level code (like C or Java) and converts it into an intermediate form, which can be easier to analyze and manipulate.
- **Portability**: This intermediate code can often run on different types of machines without needing major changes, making it more versatile.
- **Optimization:** Before turning it into machine code, the compiler can optimize this intermediate code to make the final program run faster or use less memory.

Advantages of Intermediate Code Generation

- **Easier to Implement:** Intermediate code generation can simplify the code generation process by reducing the complexity of the input code, making it easier to implement.
- **Facilitates Code Optimization:** Intermediate code generation can enable the use of various code optimization techniques, leading to improved performance and efficiency of the generated code.
- **Platform Independence:** Intermediate code is platform-independent, meaning that it can be translated into machine code or bytecode for any platform.
- **Code Reuse:** Intermediate code can be reused in the future to generate code for other platforms or languages.
- **Easier Debugging:** Intermediate code can be easier to debug than machine code or bytecode, as it is closer to the original source code.

Postfix Notation

- Also known as reverse Polish notation or suffix notation.
- In the infix notation, the operator is placed between operands, e.g., *a + b*. Postfix notation positions the operator at the right end, as in *ab +*.
- For any postfix expressions *e1* and *e2* with a binary operator *(+)* , applying the operator yields *e1e2+*.
- Postfix notation eliminates the need for parentheses, as the operator's position and arity allow unambiguous expression decoding.
- In postfix notation, the operator consistently follows the operand.

Example 1: The postfix representation of the expression (a + b) * c is : ab + c *
Example 2: The postfix representation of the expression (a - b) * (c + d) + (a - b) is : ab - cd + *ab -+

Three-Address Code

- A three address statement involves a maximum of three references, consisting of two for operands and one for the result.
- A sequence of three address statements collectively forms a three address code.
- The typical form of a three address statement is expressed as *x = y op z*, where *x*, *y*, and *z* represent memory addresses.
- Each variable *(x, y, z)* in a three address statement is associated with a specific memory location.

While a standard three address statement includes three references, there are instances where a statement may contain fewer than three references, yet it is still categorized as a three address statement.
Example: The three address code for the expression a + b * c + d : T1 = b * c T2 = a + T1 T3 = T2 + d; T 1 , T2 , T3 are temporary variables.

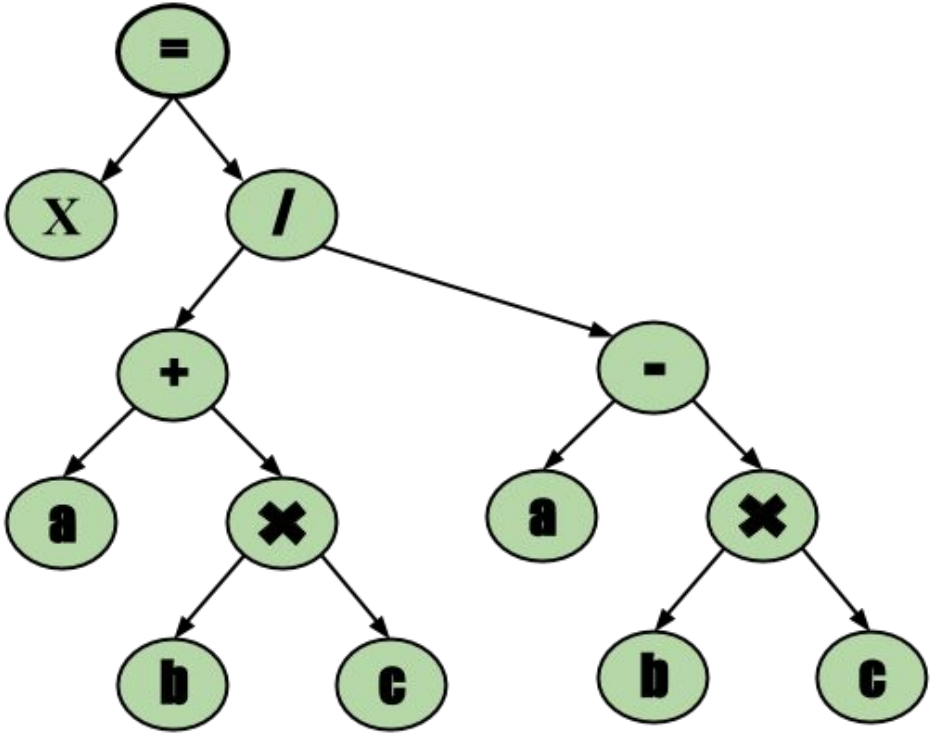
Syntax Tree

- A syntax tree serves as a condensed representation of a parse tree.
- The operator and keyword nodes present in the parse tree undergo a relocation process to become part of their respective parent nodes in the syntax tree. the internal nodes are operators and child nodes are operands.
- Creating a syntax tree involves strategically placing parentheses within the expression. This technique contributes to a more intuitive representation, making it easier to discern the sequence in which operands should be processed.

The syntax tree not only condenses the parse tree but also offers an improved visual representation of the program's syntactic structure,

$$X = (a + (b * c)) / (a - (b * c))$$

Operator Root



Control Flow Graph

A Control Flow Graph visually represents the control flow of a program, illustrating how the program moves through different statements and decision points

How to Draw a Control Flow Graph?

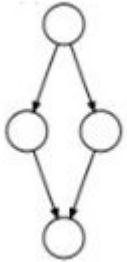
Following are the simple rules that we can follow to draw a CFG.

- Each statement will be a node in the graph
- Every node will have a directed edge either going to it or coming from it, or both.
- The entry node (first statement) will only have outgoing edges, and the exit node will only have incoming edges.
- Only conditional statements like if/else if, switch, and loops will have more than one outgoing edge.
- All paths from a node will converge at some point, with the worst case being convergence at the exit.

Example

The following are examples of control-flow graphs –

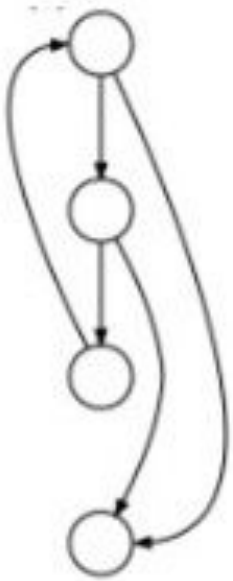
If then else – One path is taken when the condition is true (the "then" part), and another path is taken when the condition is false (the "else" part).



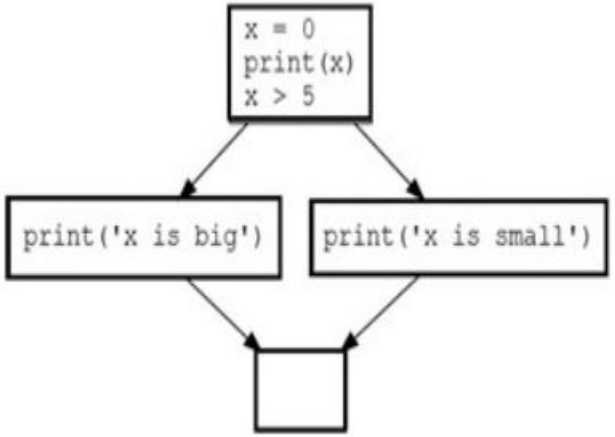
while loop – lets look at how a while loop is represented using a control flow graph. If it's true, it keeps doing the same set of actions (loop body). When the condition becomes false, the loop ends, and the next part of the program continues

natural loop with two exits – while with an if...break in the middle; non-structured but reducible.

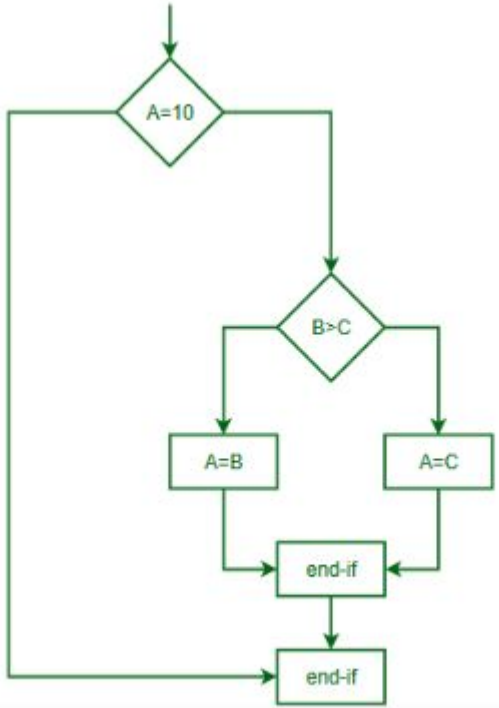
An irreducible CFG – a loop with two entry points, e.g. goto into a while or for loop.

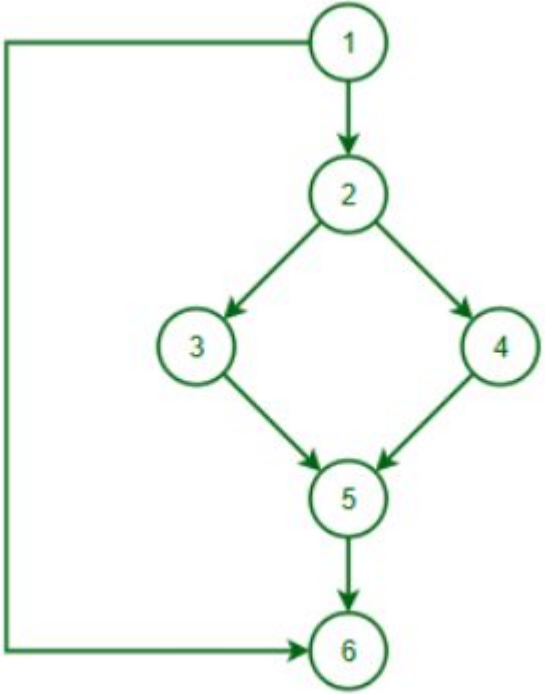


```python
x = 0
print(x)
if x > 5:
    print('x is big')
else:
    print('x is small')
```



```
if  A = 10 then
  if B > C
     A = B
  else A = C
  endif
  endif
print A, B, C
```

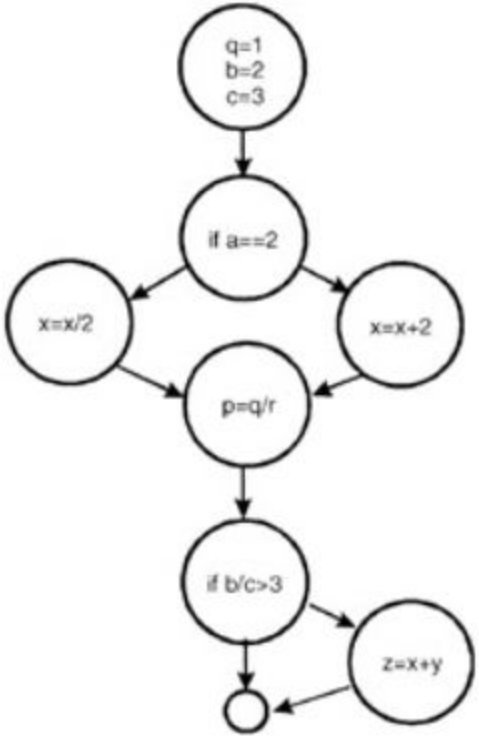

A=10
B>C
A=B
A=C
end-if
end-if


1
2
3
4
5
6

Advantage of CFG

- **Visualizes program flow:** Easy to see how a program runs.
- **Helps find errors:** Detects unreachable code or infinite loops.
- **Useful for optimization:** Improves program performance.
- **Aids testing:** Ensures all parts of the code are tested.

```
q=1;
b=2;
c=3;
if (a==2) {x=x+2;}
else {x=x/2;}
p=q/r;
if (b/c>3) {z=x+y;}
```


q=1
b=2
c=3
if a==2
x=x/2
x=x+2
p=q/r
if b/c>3
z=x+y

Symbol Table in Compiler

- Every compiler uses a symbol table to track all variables, functions, and identifiers in a program.
- It stores information such as the name, type, scope, and memory location of each identifier.\
- the symbol table supports error checking, scope management, and code optimization for runtime efficiency.
- It plays a crucial role in ensuring the correct usage of identifiers according to language rules.

Role of Symbol Table

The symbol table acts as a bridge between the analysis and synthesis phases of the compiler. It collects information during the analysis phases and utilizes it during the synthesis phases to generate efficient code, ultimately enhancing compile-time performance.

It is used by various phases of the compiler as follows:-

- **Lexical Analysis:** Creates new table entries in the table, for example, entries about tokens.
- **Syntax Analysis:** Adds information regarding attribute type, scope, dimension, line of reference, use, etc in the table.
- **Semantic Analysis:** Uses available information in the table to check for semantics i.e. to verify that expressions and assignments are semantically correct(type checking) and update it accordingly.
- **Intermediate Code Generation:** Refers to symbol table for knowing how much and what type of run-time is allocated and table helps in adding temporary variable information.
- **Code Optimization:** Uses information present in the symbol table for machine-dependent optimization.
- **Target Code generation:** Generates code by using the address information of the identifier present in the table.
- **Symbol Table entries -** Each entry in the symbol table is associated with attributes that support the compiler in different phases.

Consider this C-like code:

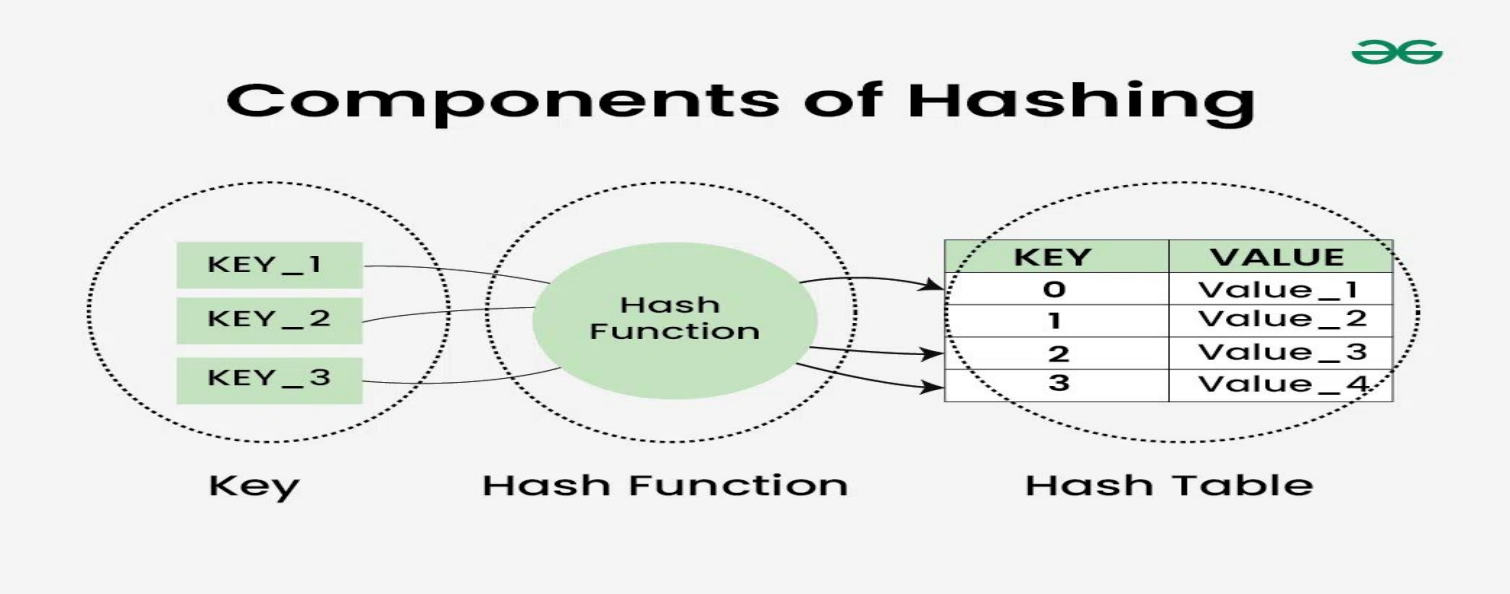
```c
int x;
float y;
x = 10;
y = 20.5;
```

The symbol table might look like:

Name	Type	Scope	Memory Location
x	int	global	0x0010
y	float	global	0x0014

What is Hash Table?

A Hash table is defined as a data structure used to insert, look up, and remove key-value pairs quickly. It operates on the hashing concept, where each key is translated by a hash function into a distinct index in an array. The index functions as a storage location for the matching value. In simple words, it maps the keys with the value.



Que - 2. The keys 12, 18, 13, 2, 3, 23, 5 and 15 are inserted into an initially empty hash table of length 10 using open addressing with hash function h(k) = k mod 10 and linear probing. What is the resultant hash table?

Unit 5: Code Generation

Code generation can be considered as the final phase of compilation.

The code generated by the compiler is an object code of some lower-level programming language, for example, assembly language.

minimum properties:

It should carry the exact meaning of the source code.
It should be efficient in terms of CPU usage and memory management.

Code generation in compiler design is the **final phase** of a compiler where the **intermediate representation (IR)** of source code is translated into **target machine code** (also called object code or assembly code), which can be executed by a computer.

Purpose of Code Generation

The goal is to produce **efficient and correct machine code** that faithfully performs the operations described in the original high-level program.

Steps Involved in Code Generation

1. **Input**: Intermediate code (such as three-address code, abstract syntax tree, or control flow graph).
2. **Register Allocation**: Assigning program variables to a limited number of CPU registers.
3. **Instruction Selection**: Choosing appropriate machine instructions for each intermediate operation.
4. **Instruction Scheduling**: Ordering instructions to improve performance and avoid pipeline stalls.
5. **Target Code Emission**: Emitting actual machine or assembly instructions.

Consider a simple arithmetic operation:

```c
a = b + c;
```

- **Intermediate Code:**

```ini
t1 = b + c
a = t1
```

- **Target Code (x86 assembly):**

```css
MOV R1, b
ADD R1, c
MOV a, R1
```

Issues in the design of a code generator

Input to Code Generator

The input to the code generator comes from the intermediate code generated by the compiler's front-end. This intermediate code is usually a higher-level representation of the program, like triples, quadruples, or abstract syntax trees. Along with this intermediate code, the code generator also uses information from the symbol table, which holds the addresses of variables and other data objects. One key challenge here is that the input must be free from syntactic and semantic errors, as the code generator assumes that proper type-checking and other error checks have already been handled by the front-end. Handling the input correctly is crucial for generating the correct target code.

Target Program

The target program is the final output of the code generator, which can be in the form of absolute machine language, relocatable machine language, or assembly language. Each type of output has its own set of challenges:

- **Absolute Machine Language** is easy to execute but lacks flexibility because it is bound to specific memory locations.
- **Relocatable Machine Language** allows parts of the program to be moved around in memory, making it suitable for linking multiple modules, but it requires a linking loader and has some overhead.
- **Assembly Language** is symbolic and needs an additional step (an assembler) to convert it into machine code, but it makes the code generation process easier.

Memory Management

Memory management in the code generation phase involves mapping variable names to their corresponding memory locations. The code generator works closely with the front-end to access the symbol table, where memory addresses for variables are stored. A major challenge is ensuring that the code generator uses memory efficiently, avoids memory conflicts, and correctly handles dynamic memory allocation. This requires careful handling of variable storage, particularly for dynamically allocated objects or large data structures, such as arrays or objects in object-oriented languages.

Register Allocation Issues

Efficient use of registers is important because registers are faster than memory, and utilizing them effectively can significantly improve program performance. The challenge lies in selecting the right variables to store in registers at different points in the program.

Register allocation involves two stages:

1. **Register Allocation:** It is selecting which variables will reside in the registers at each point in the program
2. **Register Assignment:** Assigning specific registers to those variables selected in Register Allocation.

Basic Blocks

Basic Block is a straight line code sequence that has no branches in and out branches except to the entry and at the end respectively. Basic Block is a set of statements that always executes one after other, in a sequence.

A **basic block** is a **sequence of instructions** in a program that has:

- **One entry point**: The first instruction is the only way to enter the block.
- **One exit point**: The control always leaves from the last instruction—no branches except at the end.

Properties:

- No jump **into** the middle of a block.
- No jump **out** from the middle—only at the end.
- If the first instruction executes, **all** others will execute in order.

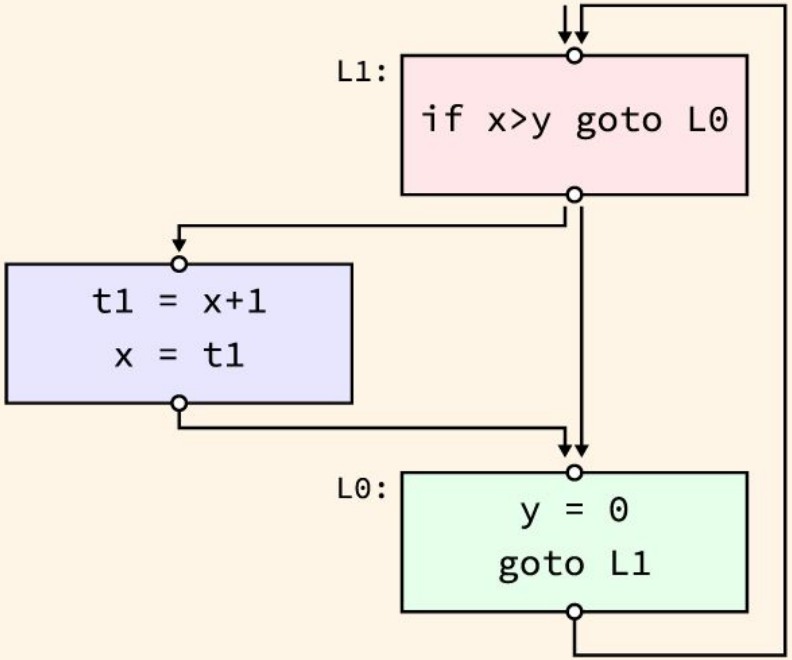
Intermediate Code:

```markdown
1. t1 = a + b
2. t2 = t1 * c
3. if t2 > 10 goto L1
4. t3 = t2 - d
5. goto L2
6. L1: t3 = t2 + d
7. L2: print t3
```

This code can be broken into **basic blocks**:

- **B1**: 1–2
- **B2**: 3
- **B3**: 4–5
- **B4**: 6
- **B5**: 7

Basic Block in Compiler Design



Flow Graph (Control Flow Graph or CFG)

A **flow graph** is a **graphical representation** of the control flow between basic blocks in a program.

- **Nodes**: Represent basic blocks.
- **Edges**: Represent possible control flow from one block to another.

B1

```
w = 0;
x = x + y;
y = 0;
if( x > z)
```

B2

```
y = x;
x++;
```

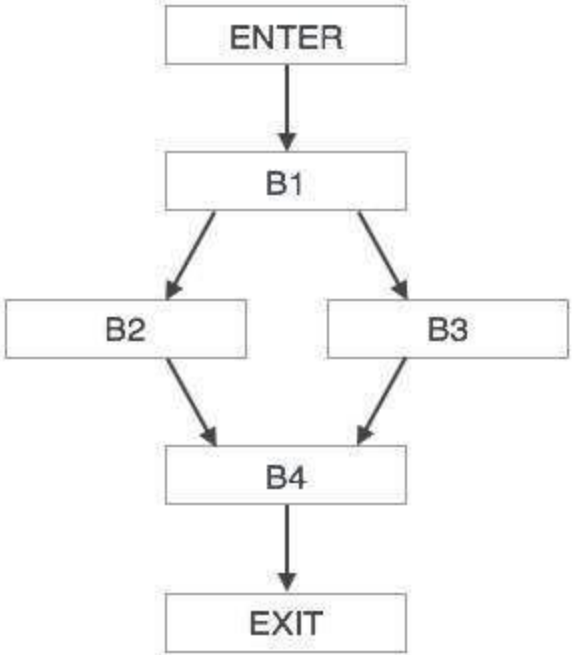
B3

```
y = z;
z++;
```

B4

```
w = x + z;
```

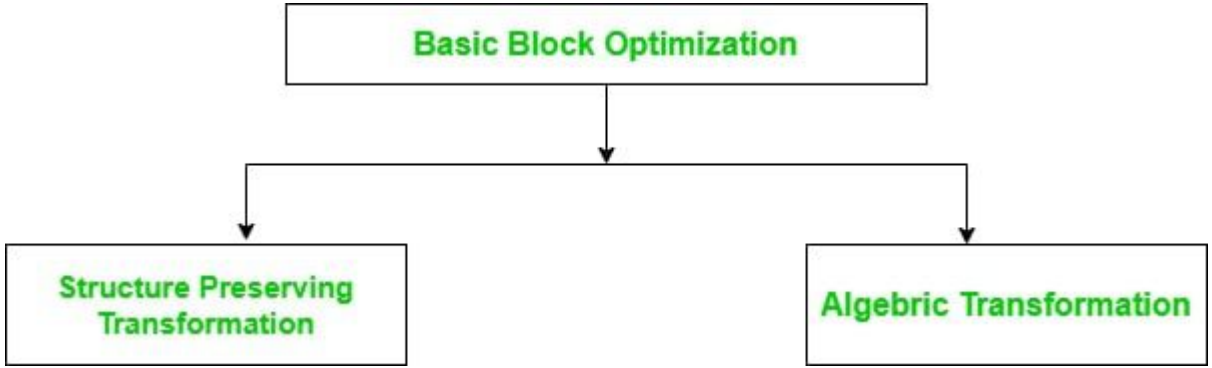
Basic Blocks



Flow Graph

Optimization of Basic Blocks

Optimization is applied to the basic blocks after the intermediate code generation phase of the compiler. Optimization is the process of transforming a program that improves the code by consuming fewer resources and delivering high speed. In optimization, high-level codes are replaced by their equivalent efficient low-level codes. Optimization of basic blocks can be machine-dependent or machine-independent. These transformations are useful for improving the quality of code that will be ultimately generated from basic block.



Structure-Preserving Transformations:

The structure-preserving transformation on basic blocks includes:

1. Dead Code Elimination
2. Common Subexpression Elimination
3. Renaming of Temporary variables
4. Interchange of two independent adjacent statements

1.Dead Code Elimination:

Dead code is defined as that part of the code that never executes during the program execution. So, for optimization, such code or dead code is eliminated.

Example:

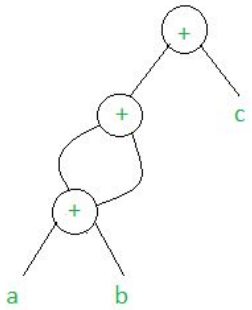
```
// Program with Dead code
int main()
{
    x = 2
    if (x > 2)
      cout << "code"; // Dead code
    else
      cout << "Optimization";
    return 0;
}
```

```
// Optimized Program without dead code
int main()
{
    x = 2;
    cout << "Optimization"; // Dead Code Eliminated
    return 0;
}
```

Common Subexpression Elimination:

In this technique, the sub-expression which are common are used frequently are calculated only once and reused when needed. DAG (Directed Acyclic Graph) is used to eliminate common subexpressions.

x=(a+b)+(a+b)+c



6 Nodes
6 Edges

DAG Representation

Before:

```
c

t1 = a + b;
t2 = a + b;
```

After:

```
c

t1 = a + b;
t2 = t1;
```

Renaming of Temporary Variables:

Statements containing instances of a temporary variable can be changed to instances of a new temporary variable without changing the basic block value.

Example: Statement t = a + b can be changed to x = a + b where t is a temporary variable and x is a new temporary variable without changing the value of the basic block.

Interchange of Two Independent Adjacent Statements:
If a block has two adjacent statements which are independent can be interchanged without affecting the basic block value.

Algebraic Transformation:

Countless algebraic transformations can be used to change the set of expressions computed by a basic block into an algebraically equivalent set. Some of the algebraic transformation on basic blocks includes:

1. Constant Folding
2. Copy Propagation
3. Strength Reduction

1. Constant Folding:

Solve the constant terms which are continuous so that compiler does not need to solve this expression.

Example:

```
x = 2 * 3 + y   ⇒ x = 6 + y  (Optimized code)
```

2. Copy Propagation:

It is of two types, Variable Propagation, and Constant Propagation.

Variable Propagation:

$x = y$ $\Rightarrow z = y + 2$ *(Optimized code)*

$z = x + 2$

Constant Propagation:

$x = 3$ $\Rightarrow z = 3 + a$ *(Optimized code)*

$z = x + a$

3. Strength Reduction:

Replace expensive statement/ instruction with cheaper ones.

```
x = 2 * y (costly)  ⇒  x = y + y (cheaper)
x = 2 * y (costly)  ⇒ x = y << 1 (cheaper)
```

Peephole Optimization

Peephole optimization is a method used to optimize a small set of compiler-generated instructions; this small set is referred to as the peephole optimization or window in compiler design.

The key points about peephole optimization are listed below –

1. It is applied after the source code is converted into the target code.
2. Peephole optimization falls under machine-dependent optimization. Machine-dependent optimization takes place after the target code is generated and adapted to the machine's architecture. It uses CPU registers and may use absolute memory references instead of relative ones.
3. It is applied repeatedly to small sections of code.

Goals of Peephole Optimization

The goals of peephole optimization include –

1. Improving performance
2. Reducing memory footprint
3. Reducing code size.

Working of Peephole Optimization in Compiler Design

Step 1: Identify the Peephole

In the first step, the compiler finds the small sections of the generated code that need optimization. Peephole refers to an instruction within a fixed window size, so the window size depends on the specific optimization being performed. The compiler helps to define the instructions within the window.

Step 2: Apply the Optimization Rule

After identification, in the second step, the compiler applies a predefined set of optimization rules to the instructions in the peephole. The compiler will search for the specific pattern of instructions in the window. There can be many types of patterns, such as insufficient code, series of loads and stores, or complex patterns like branches

Step 3: Evaluate the Result

After the pattern is identified, the compiler makes the changes in the instructions. The compiler then cross-checks the code to determine if the changes improved the code. It checks the improvement based on size, speed, and memory usage.

Step 4: Repeat

The above steps will go on in a loop by finding the peephole repeatedly until no more optimization is left in the code. The compiler will go to each instruction one at a time, make the changes, and reanalyze it for the best result.

Objectives of Peephole Optimization in Compiler Design

Increasing code speed – Peephole optimization aims to enhance the execution speed of the generated code by removing redundant or unnecessary instructions.

Reduced code size: – Peephole optimization works to reduce the size of the generated code by replacing long sequences of instructions with shorter ones.

Getting rid of dead code – Peephole optimization targets the removal of dead code, such as unreachable code, redundant assignments, or constant expressions that have no impact on the program's output.

Simplifying code – Peephole optimization aims to make the generated code more understandable and manageable by eliminating unnecessary complexities.

Peephole Optimization Techniques

Constant Folding

Constant folding is a peephole optimization technique that involves evaluating constant expressions during compile-time rather than at run-time. This can improve performance by reducing the number of computations needed during execution.

Initial Code

```
int x = 10 + 5;
int y = x * 2;
```

Optimized Code

```
int x = 15;
int y = x * 2;
```

Strength Reduction

Strength reduction is a peephole optimization technique that replaces computationally expensive operations with simpler, cheaper ones, enhancing program performance.

Redundant Load and Store Elimination

Redundant load and store elimination is a peephole optimization technique that reduces unnecessary memory accesses in a program. It eliminates repeated memory operations by reusing already loaded values.

Null Sequences Elimination

Null sequence elimination is a peephole optimization technique that removes unnecessary instructions from a program. It identifies and eliminates sequences of instructions that do not affect the final output of the program.

Register Allocations in Code Generation

Registers are the fastest locations in the memory hierarchy. But unfortunately, this resource is limited. It comes under the most constrained resources of the target processor. Register allocation is an complex problem. However, this problem can be reduced to graph coloring to achieve allocation and assignment. Therefore a good register allocator computes an effective approximate solution to a hard problem.

The register allocator determines which values will reside in the register and which register will hold each of those values. It takes as its input a program with an arbitrary number of registers and produces a program with a finite register set that can fit into the target machine.

Why it's important:

- Registers are **limited** (e.g., only 16 or so in x86-64).
- Accessing memory is **much slower** than accessing registers.
- Good register allocation = **faster code**.

What does register allocation involve?

The compiler performs two main tasks:

1. **Liveness Analysis**
   It checks **which variables are "alive"** (i.e., still needed) at each point in the program.

2. **Graph Coloring**
   Register allocation is often modeled as a **graph coloring problem**:

   - Each variable is a node.
   - An edge between two nodes means the variables are alive at the same time → they can't use the same register.
   - Colors represent registers.
   - The goal: assign colors (registers) so that no two connected nodes share the same color.

What happens if registers run out?

If there are more live variables than available registers, the compiler performs:

- **Spilling**: Moves some variables from registers to **RAM** (usually the stack). Slower but necessary.

Types of Register Allocation:

1. **Global register allocation**:
    - Works across **entire functions**.
    - Uses data flow analysis and graph coloring.
2. **Local register allocation**:
    - Works **within a basic block** (a straight-line piece of code).
    - Simpler, often used in older or simpler compilers.

Register Assignment

Register assignment is the step in compiler backends where **temporary variables** or **program variables** are assigned to **physical machine registers** after **register allocation** has decided what needs to go in a register.

```
1: a = 10
2: b = 20
3: c = a + b
4: d = c + 1
5: e = d + b
```

```
MOV R1, 10          ; a = 10
MOV R2, 20          ; b = 20
ADD R3, R1, R2      ; c = a + b → R3
ADD R1, R3, 1       ; d = c + 1 → R1 (reuse R1)
ADD R2, R1, R2      ; e = d + b → R2 (reuse R2)
```

Dynamic programming (DP) in **code generation**

Dynamic Programming is a method for solving complex problems by breaking them down into simpler subproblems, solving each subproblem once, and storing their solutions.

Two main approaches:

1. **Top-down (Memoization)** – recursion + caching
2. **Bottom-up (Tabulation)** – iterative, build solutions from the ground up

Why Use Dynamic Programming in Compiler Design?

In compiler design, many optimization problems involve **substructure and overlapping subproblems**, which are core characteristics of problems suited for dynamic programming. DP helps compilers make optimal decisions that balance performance, memory usage, and correctness.

Key Areas Where DP is Used in Compiler Design

1. **Instruction Scheduling**
2. **Register Allocation**
3. **Parsing (Context-Free Grammars)**
4. **Optimal Code Generation for Expression Trees**
5. **Dynamic Programming for Peephole Optimization**
6. **Code Generation for Regular Expressions / DFA Minimization**

Unit 6: Machine Independent Optimization

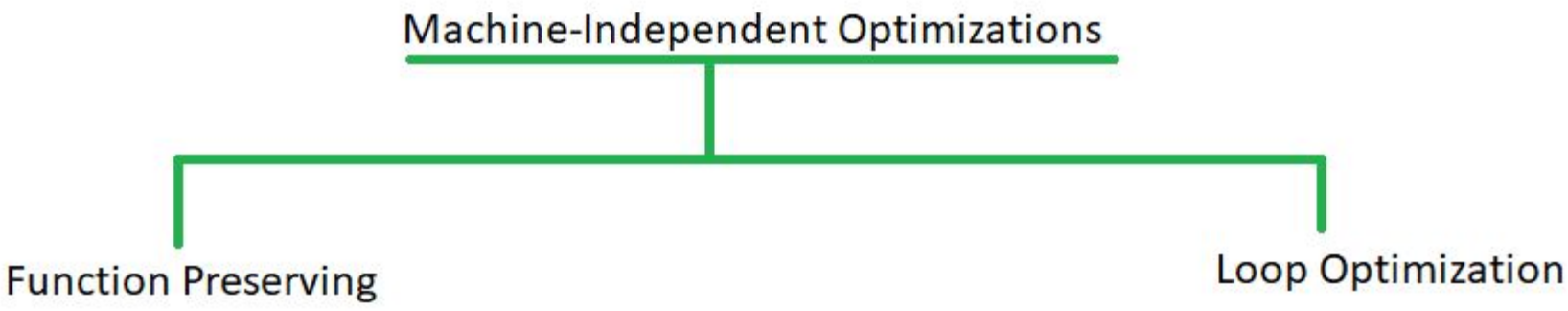
6.1 The Principal Sources of optimization

6.2 Loop in flow graph, Constant Propagation, Partial Redundancy Elimination

6.3 Introduction to Data-Flow Analysis

6.4 Solving Data-Flow Equations

Machine Independent code optimization tries to make the intermediate code more efficient by transforming a section of code that doesn't involve hardware components like CPU registers or any absolute memory location. Generally, it optimizes code by eliminating redundancies, reducing the number of lines in code, eliminating useless code or reducing the frequency of repeated code.



Function Preserving Optimization :

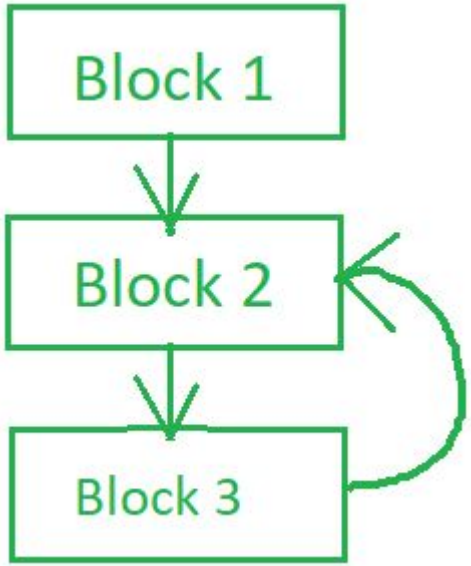
Function Preserving optimizations deals with the code in a given function in an attempt of reducing the computational time. It can be achieved by following methods:

1. Common Subexpression elimination
2. Folding
3. Dead code elimination
4. Copy Propagation

Loop Optimizations :

The program spends most of its time inside the loops. Thus the loops determine the time complexity of the program. So, in order to get an optimal and efficient code, loop optimization is required. In order to apply loop optimization, we first need to detect the loop using control flow analysis with the help of program flow graph. A cycle in a program flow graph will indicate presence of a loop. Note that, the code from Intermediate code Generation phase which is in three-address format is given as input to the optimization phase. It is difficult to identify loops in such a format, thus a program flow graph is required.

The program flow graph consists of basic blocks, which is nothing but the code divided into parts or blocks and show the execution flow of the code,



The cycle in the above graph shows a presence of loop from block 2 to block3.

Once the loops are detected following Loop Optimization techniques can be applied :

1. Frequency Reduction
2. Algebraic expression simplification
3. Strength Reduction
4. Redundancy Elimination

1. Frequency Reduction :

It applies to the concept that a loop runs for least possible lines of code. It can be achieved by following methods:

a. Code Motion :

Many times, in a loop, statements that remain unchanged for every iteration are included in the loop. Such statements are loop invariants and only result in the program spending more time inside the loop. Code motion simply moves loop invariant code outside the loop, reducing the time spent inside the loop.

```
//Before code motion

p=100
for(i=0;i<p;i++)
{
    a=b+40;          //loop invariant code
    if(p/a==0)
        printf("%d",p);
}

// After code motion

p=100
a=b+40;
for(i=0;i<p;i++)
{
    if(p/a==0)
        printf("%d",p);
}
```

```
//Before code motion
----;
while((x+y)>n)
{
    ----;
}
----;

// After code motion
----;
int t=x+y;
while(t>n)
{
    ----;
}
----;
```

b. Loop Unrolling :

If a loop runs doing the same operation for every iteration, we can perform that same operation inside the loop more than once. This is called loop unrolling. Such unrolled loop will perform the evaluation more than once in a single loop iteration.

```
//Before Loop unrolling

while(i<50)       //while loop initialize all array elements to 0;
                 //one element each iteration. Thus the loop runs 50 times.
{
    x[i]=0;
    i++;
}
```

```
//After loop unrolling

while(i<50)       //After unrolling, each iteration
                  //initializes 5 elements to 0;
                  //Thus this loop runs only 5 times.
{
    x[i]=0;
    i++;
    x[i]=0;
    i++;
    x[i]=0;
    i++;
    x[i]=0;
    i++;
    x[i]=0;
    i++;
}
```

c. Loop Jamming :

Combining the loops that carry out the same operations is called as loop jamming.

```
//Before loop jamming
----;
    for(i=0;i<5;i++)      //Setting all elements of 2D array to 0.
    {
        for(j=0;j<5;j++)
        {
            x[i][j]=0;

        }
    }
    for(i=0;i<5;i++)     //Setting diagonal elements to 1.
    {
        x[i][i]=0;


    }
----;
```

```
//After loop jamming
----;
    for(i=0;i<5;i++)      //Setting all elements of 2D array to 0
                          //and diagonal elements to 1.
    {
        for(j=0;j<5;j++)
        {
            x[i][j]=0;
            x[i][i]=1;
        }
    }
----;
```

Constant Propagation :

If any variable is assigned a constant value and used in further computations, constant propagation suggests using the constant value directly for further computations. Consider the below example

```
// Code segment
int a = 5;
int c = b * 2;
int z = a[c];

//Applying constant propagation once
int c = 5 * 2;
int z = a[c];

//Applying constant propagation second time
int c = 10;
int z = a[c];

//Applying constant propagation last time
int z = a[10];
```

4. Redundancy Elimination :

It may happen, that a specific expression is repeated in a code many times. This expression is redundant to code because we may evaluate it once and substitute its next occurrence with its evaluated value. This substitution is nothing but redundancy elimination.

```
//Code:
----;
int x=a+b;
----;
int z=a+b+40;
----;
return (a+b)*5;

//After optimization
----;
int x=a+b;
----;
int z=x+40;
----;
return x*5
```

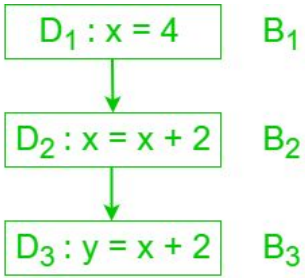
What is Data Flow Analysis?

Data flow analysis is a technique used in compiler design to analyze how data flows through a program. It involves tracking the values of variables and expressions as they are computed and used throughout the program, with the goal of identifying opportunities for optimization and identifying potential errors.

The basic idea behind data flow analysis is to model the program as a graph, where the nodes represent program statements and the edges represent data flow dependencies between the statements. The data flow information is then propagated through the graph, using a set of rules and equations to compute the values of variables and expressions at each point in the program.

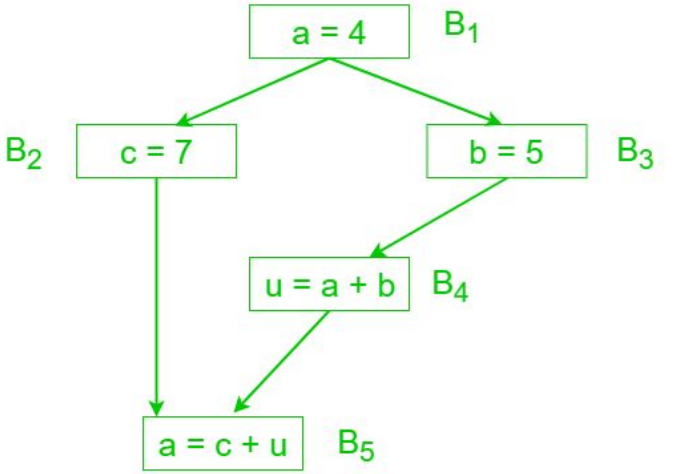
Types of Data Flow Analysis

1. **Reaching Definitions Analysis:** This analysis tracks the definition of a variable or expression and determines the points in the program where the definition "reaches" a particular use of the variable or expression. This information can be used to identify variables that can be safely optimized or eliminated.



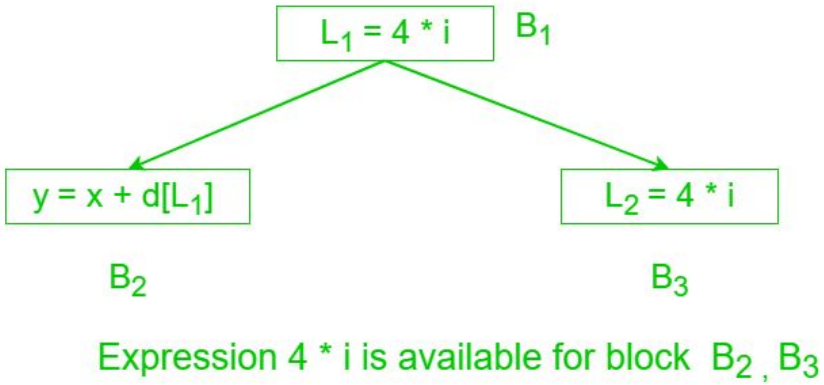
D_1 is reaching definition for B_2 but not for B_3 since it is killed by D_2

1. **Live Variable Analysis:** This analysis determines the points in the program where a variable or expression is "live", meaning that its value is still needed for some future computation. This information can be used to identify variables that can be safely removed or optimized.



a is live at block B_1 , B_3 , B_4 but killed at B_5

1. **Available Expressions Analysis:** This analysis determines the points in the program where a particular expression is "available", meaning that its value has already been computed and can be reused. This information can be used to identify opportunities for common subexpression elimination and other optimization techniques.



1. **Constant Propagation Analysis:** This analysis tracks the values of constants and determines the points in the program where a particular constant value is used. This information can be used to identify opportunities for constant folding and other optimization techniques.